

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-ada4a0148ebd96b92.jsonl`  
- SHA-256: `080e1e8e6727df1f681261b209716b26126d480ddddd151bbe220a6ac221e16e1`  
- Source modified: `2026-06-16T02:30:20+00:00`  
- Imported at: `2026-07-05T16:48:29+00:00`  
- project: `wf_b3a30c42-346`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T02:28:01.390Z)

Propose the MINIMAL design that achieves the goal: as golden-set evals grow, the GitHub repo accumulates committed structured files (JSON/CSV/etc.) that let the test suite run rally-segmentation regression WITHOUT any video ? so a contributor on another machine/clone can submit a refactor or small non-quality change and the suite catches side effects. Bias HARD toward extending the existing infrastructure named in the grounding (reuse it; do not reinvent) with the smallest possible delta. Define: the seam to freeze, the artifact files + format, where they live, the extraction step, the regression test, and how the set grows automatically as each golden video is added. Keep it shippable in one or two small PRs.

GROUNDING ? findings from codebase analysis (JSON):

```
[  
  {  
    "area": "Existing regression/golden/eval infrastructure in backend/eval (badminton rally detector) ? for a \"video-free regression from committed structured files\" design",  
    "summary": "The repo already has a mature, layered, mostly-pure eval stack under backend/eval/ that is video-free at the SCORING boundary: pure interval metrics (metrics.py, segmentation_metrics.py), one canonical GT CSV loader (gt_loader.py), a DB-read harness (harness.py), a baseline-snapshot regression gate (regression.py + run_regression.py, exit 0/1/3) and a sibling eval harness CLI (run_eval.py), an A/B experiment engine (experiment.py) with a committed leaderboard, an ablation driver (ablation.py), and two offline replay-substrate producers (fusion_golden.py -> per-video *_golden_features.json; export_wasb_segments.py). HOWEVER, none of the existing flows runs from committed-in-repo structured files end-to-end. Two distinct \"sources of truth\" exist: (a) the DB path (harness/regression read predictions from a SQLite DB that only the GPU pipeline can populate), and (b) the trajectory/feature path (calibrate_wasb, distill_local, fusion_golden, served_gate_a, ablation read WASB trajectory CSVs / golden_features JSON / .rallies.csv whose paths come from output/human_lovo_manifest.json ? which is gitignored and points at absolute paths OUTSIDE the repo). ablation.py is the closest existing thing to a video-free, file-driven regression: it loads *_golden_features.json from a directory and runs entirely on CPU with no DB and no video ? but those JSONs are not committed. The committed structured artifacts that DO survive a clean checkout live in eval_baselines/ (tracked): heuristic_lovo_n6.json (the LOVO floor), experiment_leaderboard.json, gemini_wrapper_2026-06-10.json. A design can reuse the entire scoring/loader/stats/gate stack as-is and only needs to ADD committed golden fixtures (labels + a frozen predictions/feature substrate) plus a thin file-fed runner.",  
    "keyFindings": [  
      "(1) WHAT EXISTS / HOW RUN: Two CLI entrypoints at repo root. run_eval.py (-> backend.eval.harness.evaluate) scores DB-stored predictions vs a GT CSV and prints/JSON-dumps P/R/F1 per stage; read-only, no gate. run_regression.py (-> backend.eval.regression.regress_with_provider) scores DB predictions vs GT from an annotation provider tier and compares to a snapshotted baseline JSON, returning exit 0=ok / 1=regressed / 3=no-baseline-to-compare. Both read predictions from SQLite (harness.get_predictions: 'candidates'->db.query_candidate_segments, 'final'->db.query_play_segments). backend.eval.experiment.compare/evaluate_variant is the A/B engine; backend.eval.ablation.run_ablation is the LOSO/LOGO driver (CLI: python -m backend.eval.ablation
```

--features-dir output). fusion_golden, export_wasb_segments, served_gate_a, calibrate_wasb, distill_local are all `python -m backend.eval.<mod>` CLIs. CI (.github/workflows/ci.yml) runs ONLY ruff + mypy + import-smoke + the full pytest suite (--cov-fail-under=70); it does NOT invoke run_regression/run_eval as a gate. So regression today is pytest-only (test_eval_regression.py) using inline tmp_path CSVs + in-memory Database, NEVER a committed golden run.",

"(2) ALREADY VIDEO-FREE? Partially, and in two senses. The SCORING math (metrics.py, segmentation_metrics.py) and the GT loader (gt_loader.py) are pure stdlib, no video, no GPU, no numpy. harness/regression are video-free in that they read predictions from the DB (no re-detect) ? but the DB itself can only be populated by running the GPU pipeline on video, so 'no committed file -> no run'. The CLOSEST existing video-free-from-files flow is ablation.py: load_videos(features_dir) reads every *_golden_features.json (intervals + shuttle/person/audio feature columns + golden gts baked in) and runs the full ablation 100% on CPU with NO DB and NO video. experiment.compare likewise runs purely on VideoCandidates objects (intervals+features+gts). So the ENGINE to do 'video-free regression from structured files' already exists end-to-end ? what is missing is COMMITTED inputs: the *_golden_features.json files are written to gitignored output/ and are not in the repo.",

"(3) GOLDEN FIXTURES: eval_baselines/ IS tracked (committed) ? NOT gitignored. It holds heuristic_lovo_n6.json (mean_f1 0.480 + per-video F1 floor for n=6 corpus), experiment_leaderboard.json (append-only A/B records with honest ?F1 CI + sign test), gemini_wrapper_2026-06-10.json. There is NO committed regression_baseline.json (regression.DEFAULT_BASELINE_PATH='eval_baselines/regression_baseline.json' but the file does not yet exist ? it would be committed there once created via --update-baseline). There are NO committed golden label CSVs (.rallies.csv), NO committed WASB trajectory CSVs, and NO committed *_golden_features.json. The golden corpus lives entirely OUTSIDE the repo: output/human_lovo_manifest.json (gitignored) lists 6 videos by name with absolute Windows paths like C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/*.csv and .../Golden Labelled/*.rallies.csv. Tests that need the golden corpus (test_served_gate_a_regression.py) skipif the manifest/files are absent (CI skips; owner box runs).",

"(4) ARTIFACT FORMATS ARE ALREADY STANDARDISED for a file-fed design: gt_loader.load_gt_intervals accepts BOTH golden CSV conventions through one reader (header start/end OR start_time/end_time, else positional col 0,1; MM:SS/HH:MM:SS or decimal seconds; strict raises with file:line, lenient logs skips). export_wasb_segments writes the golden/slicer schema CSV (rally_number,start_time,end_time,duration,start_mmss,end_mmss) plus a *_vs_golden comparison CSV (type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s). fusion_golden writes one JSON per video {name,fps,frame_width,candidates:[{start,end,shuttle{5 cols},person{5 cols},label}],gts:[s,e]...} consumed by ablation.load_videos and re-scorable forever with no GPU. The regression baseline JSON schema is {'<video_id>::<stage>': {video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash}} with incommensurability guards (IoU/detector/gt_hash mismatch -> flagged, not silently compared).",

"(5) FOR A 'VIDEO-FREE REGRESSION FROM COMMITTED STRUCTURED FILES' DESIGN ? REUSE vs ADD. REUSE (do not reinvent): metrics.score/match_intervals/pr_curve; segmentation_metrics.f1_at_overlaps/mean_average_precision/segment_count_ratio; gt_loader.load_gt_intervals (the one CSV reader); regression.regress/save_baseline/load_baselines/gt_hash + the exit 0/1/3 gate contract in run_regression.py; experiment.VideoCandidates + evaluate_variant + compare + record_experiment + the eval_baselines/ committed-location convention; ablation.load_videos (already reads committed-shaped golden_features JSON, DB-free, GPU-free); eval_stats CI+sign-test, splits.assert_disjoint train/eval guard. ADD: (a) commit a small golden fixture set in-repo (label CSVs in the golden schema + a FROZEN predictions/feature substrate ? either tiny *_golden_features.json or a committed predictions JSON/CSV per video) under a tracked dir (tests/ has no fixtures today; eval_baselines/ or a new tests/golden/ are the natural homes); (b) a thin file-fed predictions loader that feeds metrics/regression WITHOUT the DB (today regression.regress only takes predictions via harness.get_predictions -> DB; you need a variant that reads predictions from a committed file); (c) commit the manifest with repo-relative paths (current output/human_lovo_manifest.json is gitignored + absolute-pathed); (d) optionally wire run_regression into CI (today it is not a CI gate)."

```
],
"dataStructures": [
  {
    "name": "RegressionResult / regression_baseline.json",
    "where": "backend/eval/regression.py
(DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json ? file NOT yet committed)",
    "shapeOrFormat": "Baseline store JSON: { '<video_id>::<stage>':
```

```

{video_id,stage,precision,recall,f1,iou_threshold,detector,gt_hash} }. RegressionResult
dataclass adds regressed:bool, reasons:[str], has_baseline:bool, tolerance,
baseline_precision/recall. gt_hash = sha1 of sorted rounded GT intervals (detects label drift).
Incommensurability guards flag IoU/detector/gt_hash mismatch."
    },
    {
        "name": "Golden / slicer CSV schema",
        "where": "backend/eval/export_wasb_segments.py (SEG_FIELDS); read by
gt_loader.load_gt_intervals and tools.rally_slicer.load_rally_labels",
        "shapeOrFormat": "CSV columns:
rally_number,start_time,end_time,duration,start_mmss,end_mmss. The canonical GT reader
(gt_loader) also accepts the collector export form (start,end), positional headerless, and
MM:SS/HH:MM:SS or decimal-seconds timestamps. Comparison CSV (COMP_FIELDS):
type,ref,start_time,end_time,mmss,status,iou,pred_start,pred_end,start_err_s,end_err_s."
    },
    {
        "name": "*_golden_features.json (replay substrate)",
        "where": "produced by backend/eval/fusion_golden.py (run -> out_dir, default gitignored
output/); consumed by backend/eval/ablation.py load_videos and the experiment engine",
        "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end,
shuttle:{duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person:{players_in_co
urt,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
audio?:{audio_hit_count,audio_hit_rate,audio_hit_strength_mean}, label:int]],
gts:[{start,end},...]]. This is the GPU-free, video-free re-scoring input ? exactly the artifact
a file-driven regression would commit."
    },
    {
        "name": "human_lovo_manifest.json",
        "where": "output/human_lovo_manifest.json ? GITIGNORED (output/ is in .gitignore line
39); read by fusion_golden, served_gate_a, distill_local, calibrate_wasb CLIs",
        "shapeOrFormat": "JSON array of [{name, trajectory:<abs path to WASB CSV>, fps,
frame_width, labels:<abs path to .rallies.csv>}]. Paths are absolute and outside the repo
(C:/Users/avidu/Projects/Annotation Setup/...), so this corpus does NOT survive a clean
checkout. n=6 owned golden videos."
    },
    {
        "name": "VideoCandidates / Variant / experiment record",
        "where": "backend/eval/experiment.py; leaderboard committed at
eval_baselines/experiment_leaderboard.json",
        "shapeOrFormat": "VideoCandidates(name, intervals:[(s,e)], features:{col->[per-candidate
floats]}, gts:[(s,e)] or None). Variant = declarative re-scoring recipe
(min_crossings/min_players gates, feature_names, classifier_model, threshold, merge_gap; control
defaults are no-op). Leaderboard row: {timestamp,iou,label_set_hash,num_videos,variant_a/b:{name
,spec_hash,aggregate},delta,honest_delta_f1:{mean_delta,ci_low,ci_high,sign_test:{wins,losses,p
value}}}}."
    },
    {
        "name": "WASB trajectory CSV",
        "where": "parsed by TrackNetRunner.parse_trajectory_csv; consumed by
calibrate_wasb.make_predict_fn, export_wasb_segments, served_gate_a,
distill_local.build_candidates",
        "shapeOrFormat": "CSV columns: Frame,Visibility,X,Y (one row per frame). The per-video
raw substrate from which candidate windows + shuttle features are derived; lives outside the
repo per the gitignored manifest."
    },
    {
        "name": "split manifest (train/eval guard)",
        "where": "backend/eval/splits.py load_split/assert_disjoint",
        "shapeOrFormat": "JSON list of entries each with match_id (or id fallback) + split in
{'train','eval'}; raises on overlap. Pure-logic guard, not yet wired into
distill_local/run_regression (wiring deferred per its docstring)."
    }
],
"videoDependency": "SCORING and the existing experiment/ablation engines are fully video-
free. metrics.py, segmentation_metrics.py, gt_loader.py, splits.py, regression.py (the pure

```

parts) need no video/GPU/numpy. experiment.compare and ablation.run_ablation run end-to-end on CPU from in-memory VideoCandidates / committed-shaped *_golden_features.json ? no video, no DB, no Gemini. The DB-read path (harness.get_predictions, regression.regress_with_provider, run_eval/run_regression) is video-free at run time BUT depends on a SQLite DB that only the GPU pipeline (run on real video) can populate ? so without a committed file it cannot run in CI. The FEATURE-PRODUCING step (fusion_golden person detection over the original MP4) and the served-path windowing (served_gate_a reads cached trajectory CSVs, GPU-free with person cue OFF) need the out-of-repo golden corpus (trajectories + .rallies.csv via the gitignored manifest). Net: the design's goal is achievable with ZERO video by committing a frozen predictions/feature substrate + labels and feeding the already-pure scoring/regression layer.",

"builtVsPlanned": "BUILT & shipped (with passing pytest): pure metrics + segmentation metrics; canonical GT loader (gt_loader, closes the two-loader fork, ADR-008 prereq); DB-read harness + run_eval CLI; baseline-snapshot regression gate + run_regression CLI with exit 0/1/3 and incommensurability guards; experiment A/B engine (compare/evaluate_variant/compare_unlabeled/record_experiment) with committed leaderboard; LOSO/LOGO ablation driver with honest CI+sign-test verdicts (earns_keep/suggestive/inert/structural_protected) + PDF export; fusion_golden feature extractor (with run-telemetry black box); export_wasb_segments for eyeball verification; served_gate_a production-path litmus; calibrate_wasb + distill_local; splits train/eval guard (pure-logic). Committed fixtures: eval_baselines/{heuristic_lovo_n6.json, experiment_leaderboard.json, gemini_wrapper_2026-06-10.json}. PLANNED / NOT built: a committed regression_baseline.json (default path reserved, file absent); committed golden label CSVs / trajectories / *_golden_features.json (all currently land in gitignored output/ or outside the repo); splits wiring into distill_local/run_regression (explicitly deferred in its docstring); regression as a CI gate (CI runs only lint/mypy/import-smoke/pytest+coverage). No 'video-free regression from committed structured files' runner exists yet ? the engine exists, the committed inputs and the file-fed predictions loader do not.",

"constraints": [
"Commercial-clean licensing: every dependency code/data/weights MIT/Apache-2.0/BSD only; paid LLMs are inference/serving/fallback only, never a training data source.",
"Dev container lacks GPU/torch/cv2/numpy ? the full detect pipeline cannot run here; pure-logic eval (metrics, gt_loader, regression core, experiment, ablation, splits) DOES run here. A committed file-driven regression must stay in the pure-logic lane to be CI-runnable.",
"output/ is gitignored (.gitignore line 39) and *.db is gitignored (lines 36-38) ? so the runtime DB, the manifest, golden_features JSON, and the default regression_baseline.json location do NOT survive a clean checkout unless explicitly relocated. eval_baselines/ is the established TRACKED home for committed eval artifacts.",
"Honest small-n contract (owner directive 'NUMBERS INFORM, FOUNDATIONS DECIDE'): never a bare mean ? every signal/regression delta ships a bootstrap CI + exact paired sign test (eval_stats); weak/non-significant cues are retained default-OFF, never deleted on a small-n null; structural guards (Gate A) are never auto-demoted on 'no lift'.",
"Train/eval separation: split unit = match id (never clip/MD5); splits.assert_disjoint must guard any fixture used both for training a scorer and for regression baselines.",
"gt_hash / label_set_hash / spec_hash fingerprints must travel with any committed baseline so a comparison against changed labels or a changed recipe is flagged incommensurable rather than silently trusted (the existing regression.py and experiment.py guard contract).",
"eval must not drift from serve: distill_local.PROPOSAL vs SERVED windowing are single-sourced in backend.eval.windowing; served_gate_a pins production operating point (vel 0.02, merge 2.0, min 2.0). A file-driven golden must record which windowing/operating-point it was frozen at."],

"relevantPaths": [
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/metrics.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/segmentation_metrics.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/gt_loader.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/harness.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/experiment.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/ablation.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/fusion_golden.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/export_wasb_segments.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/served_gate_a.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/calibrate_wasb.py",

```

"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/distill_local.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/splits.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/run_eval.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/run_regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/heuristic_lovo_n6.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/experiment_leaderboard.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/eval_baselines/gemini_wrapper_2026-06-10.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_gt_loader.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_eval_harness.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_fusion_golden_telemetry.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_served_gate_a_regression.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test_export_wasb_segments.py",
"C:/Users/avidu/Projects/badminton-highlight-indexer/.github/workflows/ci.yml",
"C:/Users/avidu/Projects/badminton-highlight-indexer/.gitignore"
],
"risksGaps": [
  "No committed golden fixture survives a clean checkout for an END-TO-END regression: the n=6 corpus (trajectories + .rallies.csv) is referenced only by the gitignored output/human_lovo_manifest.json via absolute, machine-specific Windows paths; the *_golden_features.json substrate lands in gitignored output/. So a 'video-free regression from committed files' must FIRST commit a (small) frozen substrate + labels + a repo-relative manifest. eval_baselines/ is the proven tracked home.",
  "regression.regress can only obtain PREDICTIONS via harness.get_predictions, which queries the SQLite DB. There is no existing path to feed regression committed predictions from a file ? a thin 'predictions-from-file' loader (or reusing experiment.VideoCandidates -> evaluate_variant) must be ADDED so the gate runs with no DB.",
  "The two truth-sources can diverge: harness/regression score DB rows (candidate_segments / play_segments) while experiment/ablation/served_gate_a score trajectory-derived windows. A file-driven design should pick ONE substrate and record its provenance (windowing/operating-point, detector, gt_hash) to avoid an incommensurable committed baseline.",
  "Regression is NOT a CI gate today (ci.yml runs only lint/mypy/import-smoke/pytest+coverage). If the design intends CI enforcement, that wiring is net-new and must run in the no-GPU lane.",
  "splits.assert_disjoint exists but is NOT wired into regression/distill_local ? committing a golden fixture risks silent train/eval leakage unless the guard is connected when the fixture is introduced.",
  "test_served_gate_a_regression.py and the golden-corpus tests skip cleanly when files are absent; this means the production-path Gate-A litmus is effectively UNVERIFIED in CI ? a committed mini-substrate would also let that litmus (or a reduced form) actually run in CI.",
  "A committed golden corpus must respect the licensing + minors-exclusion guardrails; raw match footage cannot be committed, but derived trajectory CSVs / feature JSON / interval labels are the safe, video-free artifacts to commit."
]
},
{
  "area": "Golden-set storage, sharing, and regression model ? design + built state",
  "summary": "Golden ground truth is per-video rally [start,end) interval CSVs, keyed by file MD5 (video_id), produced by the Roora vendor (schema v8) and ingested via the sibling collector. The eval harness (metrics/regression/experiment) is structurally video-free already ? it re-scores predictions stored in SQLite and golden labels read from CSV. CRUX: NO doc proposes committing structured per-video label or feature artifacts to the repo for video-free regression. The deliberate design is the OPPOSITE: GOLDEN_DATA_SHARING.md keeps the heavy per-video artifacts (golden-label CSVs, *_golden_features.json, trajectory CSVs, manifests) OUT of git ? they live in gitignored output/ and sync between the GPU box and CPU dev sessions via a PRIVATE OneDrive folder, explicitly because they derive from the proprietary corpus and must never be committed. What IS tracked in git is only the small SCORE artifacts in eval_baselines/ (aggregate + per-video F1, deltas, honest CI/sign-test, label_set_hash, spec hashes) ? no raw labels or features. So a new proposal to commit structured per-video artifacts for video-free

```

regression would EXTEND a gap, not duplicate anything; it must reconcile with (a) the OneDrive sharing convention and (b) the licensing/privacy guardrail that keeps proprietary-derived per-video data private.",

"keyFindings": [

"(1) HOW STORED/SHARED TODAY: Golden labels = one CSV per video, one rally per row, [start,end]. Two schema layers: the minimal eval-harness format (EVALUATION.md §2: 'start,end' header optional; decimal seconds OR MM:SS/HH:MM:SS; malformed rows fail loudly) read by backend/eval/annotations.py + gt_loader.load_gt_intervals (the canonical golden reader). The richer VENDOR deliverable (ROORA_LABELING_GUIDELINES.md §1) is a per-rally CSV named exactly <video_id>.csv. Keying is by video_id = file MD5 (CLAUDE.md; EVALUATION.md §3; compute_video_id) ? the vendor is told NOT to trim/re-encode because the pipeline IDs by checksum. The golden-label naming convention in code is X.rallies.csv -> X.MP4 (fusion_golden.py::_derive_video_path).",

"(1b) LOCATION: live golden CSVs + the GPU-extracted *_golden_features.json + trajectory CSVs live in gitignored output/ (per-machine) and scattered local folders.

GOLDEN_DATA_SHARING.md (#175, 2026-06-15) wires GPU box <-> CPU dev sessions via a single PRIVATE OneDrive folder: %USERPROFILE%/OneDrive/rally-annotator/golden-data/ with subdirs features/<corpus-tag>/, trajectories/<corpus-tag>/, manifests/, baselines/. corpus-tag = e.g. 2026-06-15-n6. Raw videos are NEVER put there (kept local to GPU box). Optional ergonomics: RALLY_GOLDEN_DIR env var + scripts/sync_golden.py (in #175, not required).",

"(2) CRUX ? NO doc proposes committing structured per-video LABEL/FEATURE artifacts to the repo for video-free regression. The codebase ALREADY supports video-free regression a different way: detection runs ONCE per video, all per-cue signals + per-candidate features persist to SQLite candidate_segments.stats (JSON) and to output/*_golden_features.json, then ANY variant re-scores OFFLINE on CPU with no GPU/video (EXPERIMENT_HARNESS.md §2, §5.1; ABLATION_LAYER.md; standing lesson 'persist detection once, re-score forever'). But the persistence target is gitignored output/ + private OneDrive, NOT the repo. GOLDEN_DATA_SHARING.md is explicit: 'The shared folder lives OUTSIDE the repo -> never committed. Keep output/ gitignored... features derive from the Proprietary corpus -> keep the OneDrive folder private.' So committing per-video artifacts to git would be a NEW extension, contradicting the current private/OneDrive sharing posture unless reconciled.",

"(2b) WHAT IS COMMITTED TO THE REPO TODAY (verified via git ls-files): only eval_baselines/*.json SCORE artifacts ? experiment_leaderboard.json (per-A/B-comparison aggregate P/R/F1/mIoU + honest_delta_f1 CI + sign test + label_set_hash + variant spec_hash + promotion/revert records), heuristic_lovo_n6.json (mean + per_video F1 for the 6-match corpus), gemini_wrapper_2026-06-10.json. These contain SCORES and hashes, NOT raw rally labels or feature values. regression.py's DEFAULT_BASELINE_PATH eval_baselines/regression_baseline.json is referenced but NOT present/tracked. So the repo already commits a fingerprint+score layer (label_set_hash, gt_hash/gt_fingerprint) that ties a baseline to a label set WITHOUT storing the labels ? a natural anchor any new per-video-artifact proposal should build on.",

"(3) SCHEMA + VERSIONING (Roora): ROORA_LABELING_GUIDELINES.md is at VERSION v8 (2026-06-13), declared the SINGLE canonical copy and the sole label to use going forward; it supersedes Drive copies v3/v5/v6/v7 and an internal 'DRAFT v2' (those numbers never matched). The in-repo file is canonical because the fusion fields originate here; a sibling-collector mirror, if staged, must be regenerated from this file. Columns: rally_number, start_mmss, end_mmss, start_time(sec, auto), end_time(sec, auto), shots_count, ending_reason {winner|forced_error|unforced_error|service_fault|let|other}, last_shot_outcome {in|net|out|n_a}, score_before, score_after, notes. Plus a fusion-added per-video INTAKE MANIFEST (NOT in the per-rally CSV): video_id, fps, court_corners (4 px points), net_line (2 px endpoints), singles_or_doubles. ending_reason is shared across all net-separated sports. Three golden rules: ignore all dead time; include EVERY rally (incl. service faults/lets, EXCEPT warm-up and partial edge rallies); a shot = one contact. Quality bar: boundaries within ±0.3s, frame-grid boundaries auto-FAIL. Tier-B (per-shot timestamps, per-shot type) is appendix-only, do-not-start.",

"(4) BUILT vs PLANNED vs OWNER-GATED. BUILT: GS-1 FileProvider + AnnotationProvider registry (backend/annotations/, providers file+auto); GS-2 regression.regress()/regress_with_provider + save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle scaffold; the full eval engine (metrics.py, calibration.py LOVO, classifier.py pinnable JSON model, training.py label/feature glue); EXPERIMENT_HARNESS P1 experiment.py compare()/compare_unlabeled()/record_experiment (shipped 2026-06-13); ABLATION_LAYER Phase-0 ablation.py (LOSO/LOGO, CI+sign test, litmus test); fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus; collector import-local for 3 sports; Roora v8 guideline. PLANNED/owner-gated: GS-3 HumanVendorProvider (parked on Roora pilot confirming quality); GS-5 trigger wiring (CI gate/scheduled); EXPERIMENT_HARNESS P2 per-cue flags+feature persistence, P3 CI promotion gate;

PER_VIDEO_OUTPUT_LAYOUT (design only, not started); internal gold key still pending. GATE-A: PROMOTED then REVERTED (#146->#151, eval!=serve) ? now OFF-by-default, opt-in only.",

"(5) STATED CONSTRAINTS ON SHARING GOLDEN DATA. Licensing/privacy: golden sets built ONLY from licensed/owned footage (founder's collection + ShuttleSet); NO end-user uploads ever flow to a vendor; all third-party video egress through ONE provider boundary with DPA/no-reuse/delete-on-delivery/access-logging (GOLDEN_SET_VENDORING.md §2,§4). Repo: per-video golden artifacts are proprietary-derived -> kept OUT of git, private OneDrive only, no public share links (GOLDEN_DATA_SHARING.md Guardrails). Minors: EXCLUDED ENTIRELY from the labeled pool ? footage with children is not annotated, flagged back; minors-exclusion enforced at the pooling query (ROORA v8 §8 data-handling; QUALITY_ITERATIONS personalization decisions; CLAUDE.md). Biometrics/identity: no biometric/identity labeling, players referred to by per-video pseudonyms only, no cross-video person ID, no gait, no pose/skeleton/keypoint; work-for-hire IP assignment of labels (ROORA v8 §8). Train/eval leakage: split by match/session never by clip; hold out whole conditions; train+eval in separate catalogs with HARD ingest/export guardrails still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)."

```
],
"dataStructures": [
  {
    "name": "Golden rally label CSV (eval-harness minimal format)",
    "where": "docs/EVALUATION.md §2; read by backend/eval/annotations.py +
gt_loader.load_gt_intervals; naming X.rallies.csv (fusion_golden.py)",
    "shapeOrFormat": "One CSV per video. Header 'start,end' optional/auto-detected. One
rally per row: start,end as decimal seconds (102.5) OR MM:SS/HH:MM:SS (00:01:12), forms mixable.
Blank lines and # comments ignored. Malformed rows (bad ts, start>=end) fail loudly. video keyed
by file MD5."
  },
  {
    "name": "Roora vendor per-rally CSV (schema v8)",
    "where": "docs/ROORA_LABELING_GUIDELINES.md §1; ingested via sibling collector python -m
src.cli.curate import-local",
    "shapeOrFormat": "One CSV per video named <video_id>.csv, UTF-8, header required, one
row per rally chronological. Cols: rally_number(int 1-based), start_mmss(m:ss.mmm), end_mmss,
start_time(sec auto), end_time(sec auto), shots_count(int),
ending_reason(winner|forced_error|unforced_error|service_fault|let|other),
last_shot_outcome(in|net|out|n_a), score_before, score_after, notes(required when other). Rules:
end>start; no overlaps end[N]<=start[N+1]; ±0.3s boundary tolerance."
  },
  {
    "name": "Per-video intake manifest (fusion-added)",
    "where": "docs/ROORA_LABELING_GUIDELINES.md §A; template INTAKE_MANIFEST_TEMPLATE.csv
(collector side)",
    "shapeOrFormat": "One row per video, recorded once (NOT in the per-rally CSV):
video_id(str), fps(num), court_corners(4 (x,y) px points clockwise from near-left), net_line(2
(x,y) px endpoints), singles_or_doubles(singles|doubles). Worked ex:
court_corners=[(360,980),(1560,980),(1330,300),(590,300)], net_line=[(470,640),(1450,640)]."
  },
  {
    "name": "*_golden_features.json (replayable offline re-scoring substrate)",
    "where": "written by backend/eval/fusion_golden.py to gitignored
output/<name>_golden_features.json; shared via OneDrive features/<corpus-tag>; NOT committed to
git",
    "shapeOrFormat": "Per-video JSON record: {name, fps, frame_width, candidates:[{start,
end, shuttle{5 fps/res-invariant feats:
duration,peak_velocity,mean_velocity,active_ratio,net_crossings}, person{5 feats:
players_in_court,two_sided_motion,mean_player_motion,in_court_ratio,shuttle_player_colocation},
label(int via IoU match to GT)}], gts}. Re-scored forever on CPU by fusion_compare/ablation ? no
GPU/video."
  },
  {
    "name": "candidate_segments table (SQLite persistence substrate)",
    "where": "backend/infrastructure/database.py (add_candidate_segment :318,
query_candidate_segments :355); output/<db_name>",
    "shapeOrFormat": "Per-candidate row: video_id, detector, start_time, end_time,
chunk_index, confidence, stats(JSON ? per-cue features), detection_params(JSON), human_label,
confirmed_segment_id. The persist-once/re-score-forever substrate; keyed by video_id (file
```

```

MD5)."
    },
    {
        "name": "eval_baselines/experiment_leaderboard.json (TRACKED in git ? scores only)",
        "where": "eval_baselines/experiment_leaderboard.json; written by
experiment.record_experiment",
        "shapeOrFormat": "JSON array of comparison/promotion records: {timestamp, iou,
label_set_hash, num_videos,
variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta:{...}, honest_delta
_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test:{wins,losses,ties,n_effective,p_valu
e}}, optional promotion:{cue,promoted_to,decision,reverted,...}}. Contains SCORES + hashes, NO
raw labels/features."
    },
    {
        "name": "regression baseline.json (per-video score snapshot)",
        "where": "backend/eval/regression.py DEFAULT_BASELINE_PATH =
eval_baselines/regression_baseline.json (referenced; not currently tracked);
eval_baselines/heuristic_lovo_n6.json is a tracked sibling",
        "shapeOrFormat": "save_baseline(video_id, stage, precision, recall, f1, ...,
gt_fingerprint). RegressionResult: regressed, reasons, gt_hash, baseline P/R, tolerance,
has_baseline. heuristic_lovo_n6.json: {name, description, computed, mean_f1, std,
per_video:{<video_name>: f1}}. Per-video keyed by video_id/name; ties a baseline to a label set
via gt_fingerprint/label_set_hash WITHOUT storing labels."
    }
],
"videoDependency": "The eval/regression machinery is ALREADY video-free at score time and
that is the explicit architecture: detection (GPU/WSL, needs the video) runs ONCE per video; all
per-cue signals + per-candidate features persist (SQLite candidate_segments.stats JSON and
output/*_golden_features.json); then fusion_compare.py / ablation.py / experiment.compare() /
run_regression.py re-score ANY number of variants on pure CPU reading only the small JSON + the
golden-label CSV ? no video, no GPU, no API calls, deterministic (EXPERIMENT_HARNESS $2,
ABLATION_LAYER, EVALUATION $intro). Golden-label CSVs themselves require no video to consume.
What is NOT yet video-free / not in the repo: the per-video artifacts (golden-label CSVs,
*_golden_features.json, trajectory CSVs, manifests) are gitignored and shared only via private
OneDrive (GOLDEN_DATA_SHARING.md), so a fresh git clone alone CANNOT run a regression ? it has
the score baselines (eval_baselines/) and the harness code but not the labels/features. A clone
CAN run the deterministic synthetic-row unit tests; the golden-litmus tests
(test_ablation.py::test_litmus_reproduces_gate_a, test_fusion_compare.py) skipif the
output/*_golden_features.json substrate is absent. So committing structured per-video artifacts
to the repo is exactly what would make regression runnable from a clone alone ? the unfilled
gap.",
"builtVsPlanned": "BUILT (shipped): GS-1 AnnotationProvider ABC+registry+FileProvider
(backend/annotations/, keys file+auto); GS-2 regression.regress()/regress_with_provider +
save/load_baseline + run_regression.py CI CLI (exit 0/1/2/3); GS-4 AutomatedProvider oracle
scaffold; eval engine
metrics.py/calibration.py(LOVO,cross_validate,improvement_report)/classifier.py(pinnable JSON
LogisticRegression)/training.py(label_candidates,build_training_set); EXPERIMENT_HARNESS P1
experiment.py compare()/compare_unlabeled()/record_experiment + JSON leaderboard (2026-06-13);
honest-stats helpers C1-C4 (eval_stats.py, score_calibration.py, segmentation_metrics.py ?
landed, NOT yet wired into default harness flow); ABLATION_LAYER Phase-0 ablation.py with litmus
test; fusion_golden.py GPU extractor -> *_golden_features.json; served_gate_a.py litmus + test;
FusionSegmenter (fusion_hybrid) default-OFF; collector import-local for 3 sports; Roora v8
guideline (in-repo canonical). PLANNED (not started): GS-3 HumanVendorProvider (parked until
Roora pilot confirms quality); GS-5 trigger wiring; EXPERIMENT_HARNESS P2 per-cue enable flags +
feature persistence, P3 CI promotion gate; PER_VIDEO_OUTPUT_LAYOUT (L0-L3, design only);
internal gold key; hard train/eval-catalog ingest/export guardrails (BACKLOG);
GOLDEN_DATA_SHARING optional RALLY_GOLDEN_DIR + sync_golden.py. OWNER-GATED / NOTABLE: GATE-A
net_crossings>=2 was PROMOTED to served default (#146) then REVERTED (#151) as an eval!=serve
regression (offline +0.074 on proposal windowing did NOT transfer to coarse served windowing,
mean served ?F1 -0.008) ? now OFF-by-default, opt-in (indexing.fusion.min_crossings=2). Person
cue ?F1 -0.021 and +audio +0.079 both fail the promotion bar at n=6 (CIs span 0) -> stay
default-OFF, retained as feature columns. Whole golden-set/owned-model implementation track is
owner-gated; corpus = 6 labelled matches in output/human_lovo_manifest.json.",
"constraints": [
    "Golden data built ONLY from licensed/owned footage (founder collection + ShuttleSet); NO

```



```

end-user uploads ever flow to a vendor (GOLDEN_SET_VENDORING §2)",
    "Per-video golden artifacts are proprietary-derived -> kept OUT of git; shared only via
PRIVATE OneDrive (no public links); output/ stays gitignored; do not symlink the shared folder
into a tracked path (GOLDEN_DATA_SHARING Guardrails)",
    "Only eval_baselines/*.json SCORE artifacts (aggregate+per-video F1, deltas, CI/sign-test,
label_set_hash, spec hashes) are committed to the repo ? NOT raw labels or feature values",
    "Minors EXCLUDED ENTIRELY from the labeled/training pool; footage with children is not
annotated and flagged back; minors-exclusion enforced at the pooling query (R00RA v8 §8;
CLAUDE.md GDPR Art.9)",
    "No biometric/identity/gait/pose/skeleton/keypoint labeling; players referenced by per-
video pseudonyms only, no cross-video person ID; labels are work-for-hire IP-assigned (R00RA v8
§8)",
    "Third-party video egress through ONE provider boundary with DPA, no-reuse, delete-on-
delivery, access logging; NDA before any footage shared (GOLDEN_SET_VENDORING §4; R00RA §8)",
    "video_id = file MD5 is the immutable key ? vendor must NOT trim/re-encode/rename source
video or the checksum (hence video_id) changes (R00RA v8 §1; EVALUATION §3)",
    "Train/eval split by match/recording-session never by clip; hold out whole conditions;
broadcast minimal in eval; HARD ingest/export guardrails so a video can't land in both catalogs
are still a TODO/BACKLOG (GOLDEN_SET_PHASE1_VIDEOS §5)",
    "Honest small-N contract: never a bare mean ? promote a cue only when ?F1 CI excludes 0
AND the sign test agrees; structural guards (Gate-A) exempt from auto-demotion (ABLATION_LAYER;
QUALITY_ITERATIONS)",
    "Experiment infra is single-tenant/local-first: a JSON leaderboard the size of
baseline.json ? explicitly NOT MLflow/W&B/a service (EXPERIMENT_HARNESS §9)",
    "Roora v8 is the SINGLE canonical guideline (supersedes Drive v3/v5/v6/v7 + internal DRAFT
v2); the in-repo file is canonical, any collector mirror regenerated from it (R00RA header)",
    "Fast-tier AutomatedProvider oracle MUST be a different model lineage than production
(else shared blind spots -> false green); a fast-tier regression is a smoke alarm that escalates
to the human tier (GOLDEN_SET_VENDORING §5; GS-4)"
],
    "relevantPaths": [
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_IMPLEMENTATION.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EVALUATION.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\EXPERIMENT_HARNESS.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\ABLATION_LAYER.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\R00RA_LABELING_GUIDELINES.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\QUALITY_ITERATIONS.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\PER_VIDEO_OUTPUT_LAYOUT.md",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\experiment_leaderboard.json",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\heuristic_lovo_n6.json",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\eval_baselines\\gemini_wrapper_2026-06-10.json",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_golden.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\experiment.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\ablation.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\annotations.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\annotations\\base.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
        "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_regression.py"
    ],
    "risksGaps": [
        "CRUX RESTATED for the caller: do NOT assume an existing 'commit per-video artifacts to

```

repo' proposal exists ? there is none. The only existing per-video-artifact sharing design is GOLDEN_DATA_SHARING.md, which deliberately keeps them OUT of git (private OneDrive). A new proposal would EXTEND this gap but must not CONTRADICT: (a) the 'never commit proprietary-derived data' guardrail, (b) the OneDrive sync convention, (c) the minors/biometric exclusions. Likely reconciliation: commit only DERIVED, de-identified, label-free or licensing-clean artifacts (e.g. the *_golden_features.json feature vectors are scale/translation-invariant counts/ratios, not raw frames or coords ? these MAY be repo-safe where raw CSVs/videos are not; confirm with owner).",

"The repo already commits a fingerprint layer (label_set_hash in experiment_leaderboard.json; gt_fingerprint/gt_hash in regression.py) that binds a score baseline to a label set without storing labels. Any new per-video-artifact-in-repo design should build on these hashes for integrity/keying rather than inventing a parallel scheme.",

"regression.py DEFAULT_BASELINE_PATH points at eval_baselines/regression_baseline.json but that file is NOT present/tracked ? run_regression.py would hit exit-3 (no-baseline) on a fresh clone. The tracked baselines are experiment_leaderboard.json + heuristic_lovo_n6.json + gemini_wrapper_*.json (different shape). A clone cannot currently run the per-video regression gate end-to-end.",

"Golden-litmus tests skipif the output/*_golden_features.json substrate is absent ? so on CI / a fresh clone the golden-litmus coverage is silently skipped; only synthetic-row unit tests run. Committing the (de-identified) feature substrate would also light up these tests in CI ? a concrete motivation for the proposal.",

"Two schema layers coexist (minimal 'start,end' eval CSV vs the rich Roora <video_id>.csv). The collector ingests the rich one; the indexer harness reads the minimal one via gt_loader. Any artifact-in-repo proposal must be explicit about which schema it freezes and how the rich->minimal projection is captured.",

"Train/eval leakage hard-guardrails (separate catalogs enforced at ingest/export) are still only a convention + BACKLOG TODO ? a video could currently land in both train and eval by mistake; relevant if committed artifacts blur the train/eval boundary.",

"Gate-A eval!=serve revert is the cautionary tale baked into the leaderboard: a number measured on the harness windowing did not transfer to the served windowing. Any regression model committed to the repo must record WHICH windowing/operating-point the baseline was measured at, or it will mispredict served behaviour."

```
]
},
{
  "area": "Pipeline data contracts and video-free seam(s) for regression",
  "summary": "The pipeline is a clean 4-stage chain: (1) DETECTION ? the WSL shuttle detector (WASB/TrackNet) reads frames and emits a per-frame trajectory CSV (`Frame,Visibility,X,Y`); (2) WINDOWING ? `TrackNetRunner.trajectory_to_action_windows` (the model-agnostic `\"brain\"`) turns the trajectory into candidate rally window dicts, persisted to `candidate_segments`; (3) ADJUDICATION/HANDOFF ? fusion gates + AI provider confirm windows into `play_segments`; (4) STITCH ? `VideoCompiler.compile_highlights` re-encodes/concat the selected `(start,end)` time-pairs into the reel. Video frames are required in exactly TWO places: the WASB/TrackNet trajectory pass (Stage 1) and the person-detector pass (fusion P2, default-OFF) ? both consume the original video by absolute frame index. EVERYTHING from the trajectory CSV onward (windowing, net-crossing Gate A, person features given boxes, candidate persistence, IoU scoring, segment selection, merge) is PURE deterministic transformation. The provider handoff (Gemini) and the final ffmpeg stitch are the only other steps that touch the media file. The single best video-free regression seam is the TRAJECTORY CSV: freeze `<key>_wasb.csv` per video and the windowing?gating?scoring chain replays with no GPU/WSL/frames. A second, complementary seam is the persisted `candidate_segments` rows (post-windowing), and for fusion the per-frame person-detection dict.",
  "keyFindings": [
    "STAGE SEQUENCE (runtime, trajectory hybrids = the production shuttle path): config load -> video_id=MD5(file) (backend/utils/hashing.py::compute_video_id, 64KB chunks) -> validators -> db.add_video (UPSERT) -> segmenter.process_video. Inside TrajectoryHybridSegmenter.process_video: P1 healthcheck gate -> P2 runner.run_predict(video,out_dir='output/<family>',video_id) writes trajectory CSV; parse_trajectory_csv -> List[TrajectoryPoint]; trajectory_to_action_windows -> candidate window dicts; _enrich_candidate_stats per window -> window_stats; db.add_candidate_segment persists each (full superset) -> P3 _select_for_handoff picks indices; for each, extract_video_chunk(reencode=True) padded clip -> provider.analyze_video(clip,prompt) -> AI segment dicts (or skip_ai => windows emitted verbatim) -> P4 db.add_play_segment + link_candidate_to_segment. Then COMPILER (separate call): db.query_play_segments -> filter by stitching.min_shot_count -> time_pairs=[(start_time,end_time)] ->
```

```

VideoCompiler.compile_highlights(filepath,time_pairs,out_name).",
    "ARTIFACT 1 ? Trajectory CSV (Stage-1 output, the natural freeze seam): file
`output/<family>/<stem>__<video_id[:12]>_wasb.csv` (WASB) or `<key>_predict.csv` (TrackNet).
Exact columns: `Frame,Visibility,X,Y` (header + order fixed; Visibility==1 means visible; X,Y
are ORIGINAL-frame pixel coords). Parsed to TrajectoryPoint{frame:int, visible:bool, x:float,
y:float}, frame-sorted. This is the complete handoff from the GPU/WSL world to the pure-Python
world.",
    "ARTIFACT 2 ? Candidate window dict (output of trajectory_to_action_windows; in-memory):
{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int (==1 for shuttle trajectory), chunk_index:Optional[int]
(==0 for whole-video trajectory pass)}. Windowing is deterministic: active frame iff visible AND
normalized per-second velocity (dist/frame_width * fps/dframes) > velocity_thresh; active frames
within merge_gap*fps grouped; windows shorter than min_window_duration dropped. fps<=0 ->30.0
and frame_width<=0 ->1280.0 silent fallbacks. visible==False resets prev (occlusion breaks the
track).",
    "ARTIFACT 3 ? candidate_segments DB row (persisted, detector-agnostic fine-tuning table;
THE second freeze seam): columns id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL(=end-start), confidence:REAL(=min(1.0,peak_velocity)), stats:JSON,
detection_params:JSON, confirmed_segment_id(FK SET NULL), human_label, notes, created_at,
user_id(NULL=local). stats JSON (base trajectory) = {peak_velocity, mean_velocity,
active_frame_count, track_id_count}. detection_params by detector: wasb={detector,velocity_thres
h,merge_gap,min_action_window_duration,weights_path,fusion_substrate?}; tracknet adds
queue_length; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector_score_t
hreshold,min_action_window_duration}.",
    "PER-SIGNAL FEATURE COLUMNS ? Gate A (net-crossings):
FusionSegmenter._enrich_candidate_stats ALWAYS adds stats['net_crossings']:float via
rally_gate.gate_windows (pure, from the trajectory; axis/net estimated once per video, cached).
Person cue (default-OFF, only if indexing.fusion.cue_flags.person): adds the 5
PERSON_FEATURE_NAMES = players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all scale/translation-invariant floats from
fusion_features.compute_person_features). Court mask = optional normalized
indexing.fusion.court_polygon; net=(net_axis,net_position). The learned-classifier shuttle
features (distill_local.FEATURE_NAMES) = duration, peak_velocity, mean_velocity, active_ratio,
net_crossings; the Gen-0 per-frame schema (backend/features/rally_seq.py FEAT_NAMES) = vis_rate,
spd_mean, spd_max, cross_rate, x_std, y_std.",
    "ARTIFACT 4 ? play_segment dict (final segment record, db.add_play_segment /
query_play_segments): {id:int, video_id, start_time:float, end_time:float, duration:float(=end-
start), server, receiver, metadata:dict, events:[...]}. Hybrid P4 metadata = {shot_count:int
(provider shuttle_exchange_count else max(3,int(dur/0.8))), shot_count_confidence:str (provider
value or 'Unknown'/'LocalNoAI'), detector:f'{family}-hybrid-{model_name}'}. Provider (AI)
segment dict consumed by P3 = {start_time, end_time, shuttle_exchange_count?,
shot_count_confidence?} (timestamps offset by clip w_start; _candidate_id stamped internally).",
    "FINAL REEL transform (VideoCompiler.compile_highlights): segments=[(start,end)] ->
_merge_overlapping (parse times, drop end<=start, sort, merge overlapping/abutting ranges so a
rally is never stitched twice) -> per-clip ffmpeg re-encode (libx264/superfast/crf22, identical
settings, optional watermark via -vf) -> `-f concat -c copy` -> output/<name>.mp4. Selection
upstream: API filters by stitching.min_shot_count (segments without shot_count are exempt) and
by req.segment_ids, then time_pairs=[(s['start_time'],s['end_time'])].",
    "TWO ALTERNATE PATHS that bypass the trajectory seam: `gemini` segmenter (pure-AI, no
P2/candidates ? chunks the whole video, uploads, provider returns segments directly to
play_segments) and `motion` (pixel-absdiff baseline with FABRICATED random metadata/events).
These two are the _FINAL_ONLY_SEGMENTERS in eval/batch.py ? they have no candidate stage and
cannot be frozen at the trajectory seam. The video-free seam analysis applies to the trajectory
hybrids (wasb_hybrid/tracknet_hybrid/fusion_hybrid), which are the rally-quality track."
],
    "dataStructures": [
        {
            "name": "Trajectory CSV",
            "where": "output/<family>/<stem>__<video_id[:12]>_wasb.csv (WASB) / <key>_predict.csv
(TrackNet); written in WSL, read by
backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
            "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` (fixed order); rows: Frame:int,
Visibility:int(1=visible), X:float px, Y:float px ->
TrajectoryPoint{frame:int,visible:bool,x:float,y:float} frame-sorted"
        }
    ],

```

```

{
    "name": "Candidate window dict",
    "where": "output of TrackNetRunner.trajectory_to_action_windows (tracknet_runner.py);
in-memory, consumed by trajectory_hybrid P2/P3",
    "shapeOrFormat": "{start:float(sec), end:float(sec), active_frame_count:int,
peak_velocity:float, mean_velocity:float, track_id_count:int(==1),
chunk_index:Optional[int](==0)}"
},
{
    "name": "candidate_segments row",
    "where": "backend/infrastructure/database.py (add_candidate_segment /
query_candidate_segments), table candidate_segments",
    "shapeOrFormat": "id, video_id, detector:str, chunk_index, start_time:REAL,
end_time:REAL, duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count[,net_crossings,+5
person feats]}, detection_params:JSON, confirmed_segment_id, human_label, notes, created_at,
user_id"
},
{
    "name": "Per-signal feature columns (fusion)",
    "where": "backend/pipeline/detectors/fusion_features.py::PERSON_FEATURE_NAMES +
rally_gate net_crossings; persisted into candidate_segments.stats by
fusion_hybrid.py::_enrich_candidate_stats",
    "shapeOrFormat": "net_crossings:float (Gate A, always); person (flag-gated):
players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation (all floats, scale/translation-invariant)"
},
{
    "name": "Person per-frame detections",
    "where": "backend/pipeline/detectors/person_detector.py::detections_per_frame (fusion
P2, needs video+GPU)",
    "shapeOrFormat": "{ 'frames': {frame_idx:int -> [(track_id:int, bbox:(x1,y1,x2,y2)),
...]}, 'fps':float, 'frame_width':float, 'frame_height':float}; frame_idx aligns 1:1 with
trajectory .frame"
},
{
    "name": "play_segment dict (final)",
    "where": "backend/infrastructure/database.py::add_play_segment / query_play_segments,
table play_segments",
    "shapeOrFormat": "{id:int, video_id, start_time:float, end_time:float,
duration:float(=end-start), server, receiver,
metadata:dict{shot_count:int,shot_count_confidence:str,detector:'<family>-hybrid-<model>'},
events:[...]}"
},
{
    "name": "AI/provider segment dict",
    "where": "provider.analyze_video output, consumed by trajectory_hybrid P3 and
gemini.py",
    "shapeOrFormat": "{start_time:float(decimal sec), end_time:float,
shuttle_exchange_count?:int, shot_count_confidence?:'Low'|'Medium'|'High'}; +_candidate_id
stamped internally; times offset by clip start"
},
{
    "name": "Final reel time_pairs",
    "where": "backend/main.py::compile_highlights -> VideoCompiler.compile_highlights
(compiler.py)",
    "shapeOrFormat": "List[Tuple[start:float, end:float]] -> _merge_overlapping (sort+merge)
-> per-clip reencode + concat -c copy -> output/<name>.mp4"
},
{
    "name": "GT / golden CSV (eval comparison)",
    "where": "backend/eval/annotations.py::load_annotations (start,end) and
calibrate_wasb.load_golden_gts (rally_number,start_time,end_time)",
    "shapeOrFormat": "generic: `start,end` 2-col (RAISES on bad row); golden/slicer:
`rally_number,start_time,end_time[,duration,start_mmss,end_mmss]` (SILENTLY skips bad rows)"
}

```

```

    }
  ],
  "videoDependency": "Video/frames are REQUIRED in exactly two compute steps, both at the
front of the trajectory-hybrid pipeline: (A) the shuttle trajectory pass ?
WasbRunner/TrackNetRunner.run_predict stages the video into WSL and runs the GPU CNN over
decoded frames to emit the `Frame,Visibility,X,Y` CSV (backend/pipeline/detectors/wasb_runner.py
+ wasb_infer.py, GPU/WSL/cv2); and (B) the person-detector pass ?
PersonDetector.detections_per_frame seeks the ORIGINAL video by absolute frame range (fusion P2,
default-OFF, torch/cv2/GPU). In addition the media file is touched (not 'frames' analysis but
read+transcode) at the AI handoff (extract_video_chunk re-encodes a padded clip and uploads to
Gemini) and at the final stitch (VideoCompiler re-encodes/concats clips with ffmpeg). EVERYTHING
ELSE is pure deterministic Python on numbers/strings: parse_trajectory_csv,
trajectory_to_action_windows (windowing brain), rally_gate (net-crossings Gate A),
fusion_features (person features given boxes ? pure, no cv2/torch), db.add_candidate_segment /
add_play_segment persistence, eval/metrics IoU scoring, _select_for_handoff gating, and
_merge_overlapping. So once the trajectory CSV (and, for fusion, the per-frame person dict)
exists, no video, GPU, WSL, ffmpeg, or cloud is needed to reproduce windowing, gating, candidate
rows, and IoU scores.",
  "builtVsPlanned": "BUILT and live: full trajectory-hybrid path (wasb_hybrid is the live
shuttle substrate), parse_trajectory_csv + trajectory_to_action_windows (pure, static, model-
agnostic), candidate_segments persistence (schema v5), play_segments, VideoCompiler stitch with
overlap-merge, rally_gate.py (Gate A net-crossings ? PURE, unit-tested), fusion_features.py
(PURE person-feature math, unit-tested), FusionSegmenter (registered fusion_hybrid, default-OFF,
two identity hooks; net_crossings always persisted, person features + served Gate A/B flag-gated
default-0/OFF), eval harness (metrics/calibration/distill_local/rally_seq_proto), the
skip_ai_handoff fully-local mode. The eval drivers ALREADY consume the trajectory CSV directly
and replay windowing/gating/scoring with no video ? i.e. the video-free seam is de-facto
operational in eval/ (calibrate_wasb.make_predict_fn parses the trajectory once and closes over
trajectory_to_action_windows; eval_partial_labels, export_wasb_segments, calibrate_local all do
this). PLANNED/owner-gated: per-video output layout (PER_VIDEO_OUTPUT_LAYOUT.md, NOT started ?
trajectory dir still hardcoded 'output/<family>'), P3 learned fusion (person features into
classifier, gated on golden labels), P4 audio/pose cues, served Gate-A-in-production promotion
(thresholds default 0). NO formal frozen-fixture regression harness keyed on the trajectory CSV
exists as a tracked test yet ? the seam is exploited ad-hoc by eval drivers but not pinned as a
video-free regression suite.",
  "constraints": [
    "Trajectory CSV columns are FIXED: `Frame,Visibility,X,Y` header + order; X,Y are
ORIGINAL-frame pixels (affine-resized back from the 512x288 detector input). A frozen fixture
must capture exactly this.",
    "Windowing needs fps + frame_width alongside the CSV (it normalizes velocity by
frame_width and converts frame->sec by fps). _probe_fps_width reads these from the video via
cv2; for a video-free replay they MUST be captured/pinned (fallbacks 30.0/1280.0 are silent and
would skew results). This is the one non-CSV input the windowing brain needs.",
    "Person features need the per-frame detection dict {frame_idx -> [(track_id,bbox)]} +
frame_width/height; frozen separately from the CSV. Their frame indices must align 1:1 with the
trajectory .frame.",
    "Candidate persistence is detector-agnostic by design (stats/detection_params JSON) ? a
frozen candidate_segments row is a valid second seam but is one transform DOWNSTREAM of the CSV
(windowing already applied), so it cannot regress the windowing brain itself.",
    "gemini and motion segmenters have NO candidate/trajectory stage (final-only) ? they
cannot use the trajectory seam; their regression must freeze provider responses or accept they
touch video.",
    "eval/metrics.match_intervals is GREEDY (not Hungarian) and deterministic; empty matches
yield 0.0 not NaN ? stable for golden assertions.",
    "License guardrail: any frozen-fixture tooling must stay MIT/Apache/BSD; WASB weights are
academic-data-trained (confirm terms before commercial deploy).",
  ],
  "relevantPaths": [
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\wasb_runner.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\base.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-

```

```

indexer\\backend\\pipeline\\detectors\\rally_gate.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\fusion_features.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\detectors\\person_detector.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\trajectory_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\fusion_hybrid.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\segmenters\\base.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\infrastructure\\database.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\pipeline\\stitcher\\compiler.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\utils\\hashing.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\metrics.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\calibrate_wasb.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\harness.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\features\\rally_seq.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\main.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\HOW_RALLY_DETECTION_WORKS.md",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\docs\\MULTI_SIGNAL_FUSION_PLAN.md"
],
"risksGaps": [
"SOURCES OF NON-DETERMINISM downstream of the trajectory-CSV seam ? these must be
controlled or excluded from frozen assertions: (1) FLOAT ? velocity =
(dist/frame_width)*(fps/dframes), median/spread in rally_gate, normalized person motion: all
float ops, platform-stable but assert with tolerance not exact equality; (2) ORDERING/THREADING
? P3 uses ThreadPoolExecutor(max_workers=ai_max_workers); ai_segments.extend(future.result())
iterates futures in submission order so it IS deterministic, but the underlying provider calls
are not (network) ? freeze at the windowing/candidate seam BEFORE P3, or stub the provider; (3)
TIMESTAMPS ? candidate_segments/play_segments rows carry created_at DEFAULT CURRENT_TIMESTAMP
and the DB autoincrement id ? exclude these columns from any golden comparison; (4) DB id NON-
DETERMINISM ? play_segment.id is AUTOINCREMENT, link_candidate_to_segment writes it ? compare on
(start_time,end_time,stats) not id; (5) ffmpeg/codecs ? the final reel bytes are NOT reproducible
across ffmpeg builds (libx264 version, NVENC vs libx264 fallback, watermark font) ? never assert
reel bytes; assert the time_pairs + merge output instead.",
"fps/frame_width are probed from the VIDEO (cv2) inside the segmenter, not stored in the
CSV ? a video-free replay that forgets to pin them silently falls back to 30.0/1280.0 and
produces different windows. The frozen fixture MUST include the (fps, frame_width) used.",
"No tracked frozen-fixture regression test currently pins the trajectory-CSV -> windows ->
candidates -> IoU chain end-to-end as a video-free suite; the seam is only exercised ad-hoc by
eval CLIs. This is the gap a regression harness would fill.",
"trajectory_to_action_windows treats visible==False as a hard track break (occlusion gap
resets prev) and silently clamps fps<=0->30 / width<=0->1280 ? a fixture with degenerate values
would mask real regressions.",
"Two segmenters (gemini, motion) bypass the seam entirely (no candidate stage); motion
additionally fabricates RANDOM metadata/events (random.sample/uniform) so it is inherently non-
deterministic and unsuitable for golden assertions.",
"The person-feature path (fusion, default-OFF) needs a SEPARATELY frozen per-frame
detection dict; cv2 frame-seek accuracy varies by codec (±a few frames on real footage), so the
captured dict must come from the same decode the fixture pins."
]
},
{
"area": "Test suite execution model, committed fixture data, mocking conventions, and the
right home for new auto-discovered per-video regression fixtures",
"summary": "Tests are pure-pytest auto-discovery (no test runner framework wrapping):
`pytest.ini` sets `testpaths = tests`, `run_all_tests.py` is just a thin `subprocess` wrapper

```

around `python -m pytest` (defaults to `-q`, passes all args through, prints a PASS/FAIL banner, returns pytest's exit code). There is NO conftest.py and NO committed CSV/JSON test fixtures under `tests/` or `backend/eval/` ? every test synthesizes its fixtures inline into pytest's `tmp\_path` (CSVs via `csv.writer`/`p.write\_text`, manifests + golden-feature JSON via `json.dump`). The only committed data files are 3 small JSONs in `eval\_baselines/` (the no-regression baselines/leaderboard, tracked deliberately) plus `config.json` and `artifacts/rally\_tal\_gen0/0.1.0/{manifest,weights}.json`. The real per-video golden corpus (trajectory CSVs, *.rallies.csv labels, *.golden_features.json, human_lovo_manifest.json) lives ONLY in the gitignored `output/` dir on the owner's box ? it is NOT committed, so any test depending on it (test_served_gate_a_regression.py) skips cleanly when absent. There IS a clean load?transform?assert pattern to follow (fusion_compare.load_videos reads *_golden_features.json), and an existing manifest-driven, skip-if-absent per-video regression test to mirror (test_served_gate_a_regression.py). New committed, auto-discovered per-video fixtures + a parametrized test should slot into a NEW fixtures dir UNDER tests/ (e.g. tests/fixtures/golden/) with the parametrized test in tests/test_*.py globbing that dir.",

"keyFindings": [

"DISCOVERY/RUN: pytest.ini (repo root) is the only pytest config ? a single line `testpaths = tests`. No pyproject [tool.pytest], no markers registered, no conftest.py anywhere in the repo. run_all_tests.py (repo root, 60 lines) defaults to `-q`, passes every CLI arg straight through to `python -m pytest`, and propagates pytest's exit code (CI/pre-PR friendly). The full-suite lane enforces a 70% coverage floor (CODE_MAP \$0); run with `python run\_all\_tests.py --cov=backend --cov-report=term-missing`.",

"SKIP/OPTIONAL-DEP CONVENTIONS: two idioms in use ? (a) `pytest.importorskip('cv2')` / `importorskip('obsreport')` at module top to skip whole modules when a heavy/optional dep is absent (CI lacks GPU/torch/cv2); (b) a module-level `\_skip = pytest.mark.skipif(not \_SPECS, reason=...)` decorator gated on whether real golden artifacts exist on disk. These are how real-video/GPU tests degrade to SKIP rather than ERROR in CI.",

"NO COMMITTED TEST FIXTURES: `Glob tests/\*\*/\*.csv`, `tests/\*\*/\*.json`, `backend/eval/\*\*/\*.csv|json` all return NOTHING. Every fixture is built inline into `tmp\_path`. CSV GT is written with `csv.writer` or `p.write\_text('start,end\\n0,10\\n...')` (test_gt_loader.py, test_eval_regression.py). Per-video golden-feature JSON is written with `json.dump({name, fps, frame\_width, candidates:[{start,end,shuttle{}},person{},label}], gts:[[a,b]]}, ...)` (test_fusion_compare.py::write_video). Manifests are `json.dump([name, trajectory, fps, frame\_width, labels]), ...)` (test_fusion_golden_telemetry.py::write_manifest).",

"COMMITTED DATA (the only tracked non-code data): eval_baselines/heuristic_lovo_n6.json (689 B, 15 lines ? the LOVO floor + per_video f1 map), eval_baselines/experiment_leaderboard.json (4.7 KB, 139 lines ? A/B variant rows with honest_delta_f1 CIs), eval_baselines/gemini_wrapper_2026-06-10.json (1.98 KB). Plus config.json (2.7 KB) and artifacts/rally_tal_gen0/0.1.0/{manifest,weights}.json (untracked here, ADR-008: manifest committed / weights gitignored). eval_baselines/ is deliberately tracked so a clean checkout still has the no-regression baselines.",

"MOCKING ? DETECTOR: injected, not patched. extract_video(spec, detector=None,...) takes an optional `detector` with a `detections\_over\_windows(video, windows, frame\_skip, progress\_cb)` duck-typed interface; tests pass a `\_StubDetector` returning canned `{frames, fps, frame\_width, frame\_height}` dicts (test_fusion_compare.py). fusion_golden.run() is tested by monkeypatching `fusion\_golden.extract\_video` wholesale.",

"MOCKING ? WSL: patch the single seam method `\_wsl\_bash` on a WasbRunner instance with `unittest.mock.patch.object(r, '\_wsl\_bash', side\_effect=[\_proc(rc,stdout,stderr), ...])`, where `\_proc` is a `types.SimpleNamespace(returncode, stdout, stderr)` stand-in for a CompletedProcess. Higher-level seams `\_stage\_video`, `\_stage\_infer\_script`, `\_wsl\_rm\_rf`, `\_run\_predict` are also patched. No real `wsl.exe` is ever invoked (test_wasb_runner.py).",

"MOCKING ? VIDEO/GPU: no real MP4 decode in the suite. fusion_golden derives the MP4 path from the labels filename (X.rallies.csv -> X.MP4) but the stubbed detector never opens it. TrackNetRunner.parse_trajectory_csv (Frame,Visibility,X,Y DictReader) is monkeypatched to return canned TrajectoryPoint lists, OR fed a tiny real CSV string. RunTelemetry (nvidia-smi/psutil) is made hermetic by monkeypatching gpu_name + injecting deterministic gpu_fn/sys_fn/clock collectors (test_fusion_golden_telemetry.py).",

"EXISTING load?transform?assert PATTERN TO FOLLOW:

backend/eval/fusion_compare.py::load_videos(features_dir) globs `\*\_golden\_features.json`, loads each into experiment.VideoCandidates, and compare() runs honest LOVO; tests assert on the resulting ?F1/CI/sign-test (test_fusion_compare.py). This is the canonical 'load artifact -> run transform -> assert' shape, but today the artifacts are written to tmp_path inside the test, never committed.",

"EXISTING PER-VIDEO REGRESSION TEST TO MIRROR: tests/test_served_gate_a_regression.py is a

manifest-driven, per-video, skip-if-absent regression litmus. It resolves MANIFEST from ``<repo_root>/output/human_lovo_manifest.json`` (env-overridable via `SERVED_GATE_A_MANIFEST`), `_golden_specs()` keeps only specs whose trajectory+labels files exist, and `_skip = skipif(not _SPECS, ...)` skips the whole module in CI where the gitignored corpus is absent. It loops over each golden video and asserts served-path F1/over-seg invariants. This is the closest template for new per-video regression fixtures ? the gap a committed-fixture approach would close is that it currently CANNOT run in CI (corpus uncommitted).",

"GT LOADER CONTRACT (load-bearing for fixtures):

backend/eval/gt_loader.py::load_gt_intervals reads BOTH golden-CSV conventions ? header ``start,end`` (collector curate export) OR ``start_time,end_time`` (rally-annotator .rallies.csv, extra cols ignored), else positional cols 0,1; timestamps accept decimal seconds or MM:SS/HH:MM:SS; strict=True raises with file:line, strict=False skips+logs. Any committed CSV fixture must conform to one of these two shapes."

```

],
"dataStructures": [
  {
    "name": "Per-video golden-feature JSON (*_golden_features.json)",
    "where": "Written by backend/eval/fusion_golden.py::extract_video/run into gitignored output/; consumed by backend/eval/fusion_compare.py::load_videos; synthesized inline in tests/test_fusion_compare.py::write_video",
    "shapeOrFormat": "{\n  \"name\": str,\n  \"fps\": float,\n  \"frame_width\": float,\n  \"candidates\": [\n    {\n      \"start\": float,\n      \"end\": float,\n      \"shuttle\": {5 SHUTTLE_FEATURE_NAMES: float},\n      \"person\": {5 PERSON_FEATURE_NAMES: float},\n      \"label\": int(0|1)},\n    ]\n  \"gts\": [[start,end], ...].\n  Tiny (a handful of candidates per video in synthetic tests). This is the natural unit for a committed per-video regression fixture."
  },
  {
    "name": "LOVO manifest (human_lovo_manifest.json)",
    "where": "Lives only in gitignored output/; consumed by fusion_golden.run, served_gate_a, calibrate_local; synthesized inline in tests/test_fusion_golden_telemetry.py::write_manifest",
    "shapeOrFormat": "JSON array of specs: [\n  {\n    \"name\": str,\n    \"trajectory\": \"<name>.csv\",\n    \"fps\": float,\n    \"frame_width\": float,\n    \"labels\": \"<name>.rallies.csv\",\n    optional\n    \"video_path\": str\n  }\n]"
  },
  {
    "name": "Trajectory CSV (TrackNet/WASB predict output)",
    "where": "backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv (DictReader); referenced by manifest 'trajectory' field; only exists in gitignored output/",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y`; one row per frame; Visibility==1 means shuttle visible. Parsed into TrajectoryPoint(frame:int, visible:bool, x:float, y:float).",
  },
  {
    "name": "Golden GT / rally-label CSV (two conventions)",
    "where": "backend/eval/gt_loader.py::load_gt_intervals (canonical), annotations.load_annotations, calibrate_wasb.load_golden_gts; synthesized inline in test_gt_loader.py / test_eval_regression.py",
    "shapeOrFormat": "Either header `start,end` (collector export) OR `rally_number,start_time,end_time,ending_reason,sport` (rally-annotator .rallies.csv), OR headerless positional cols 0,1. Timestamps: decimal seconds or MM:SS/HH:MM:SS. -> List[(start:float, end:float)]"
  },
  {
    "name": "Eval baseline / leaderboard JSON",
    "where": "Committed in eval_baselines/; written by backend/eval/experiment.py::record_experiment + regression.save_baseline; read by run_regression.py no-regression gate",
    "shapeOrFormat": "heuristic_lovo_n6.json: {name, description, computed, mean_f1, std, per_video:{vid:f1}}. experiment_leaderboard.json: array of {timestamp, iou, label_set_hash, num_videos, variant_a/b:{name,spec_hash,aggregate:{precision,recall,f1,mean_iou}}, delta, honest_delta_f1:{mean_delta,ci_low,ci_high,n,ci_excludes_zero,sign_test}}"
  },
  {
    "name": "WSL CompletedProcess stand-in (_proc)",
    "where": "tests/test_wasb_runner.py",
  }
]

```



```

"shapeOrFormat": "types.SimpleNamespace(returncode:int, stdout:str, stderr:str) ? fed
via patch.object(runner, '_wsl_bash', side_effect=[...]) to mock every WSL bash call"
}
],
"videoDependency": "No test in the suite decodes a real video. The pipeline cannot run here
(dev container lacks GPU/torch/cv2/numpy per CLAUDE.md). Real-video/GPU paths are handled three
ways: (1) inject a stub detector with a duck-typed detections_over_windows() so no MP4 is
opened; (2) monkeypatch TrackNetRunner.parse_trajectory_csv / fusion_golden.extract_video to
return canned data; (3) for tests that genuinely need the real golden corpus (trajectory CSVs +
.rallies.csv labels + MP4s), gate the whole module behind pytest.importorskip('cv2') and/or a
skipif on whether the gitignored output/human_lovo_manifest.json and its referenced files exist
on disk ? these SKIP in CI and only run on the owner's box. The real 6-video golden corpus
(adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy,
testlarge_short/largetest_short, mahadevpura_singles) is NOT committed; only their derived
*_golden_features.json sit in gitignored output/.",
"builtVsPlanned": "BUILT: full pytest auto-discovery suite (~85 test_*.py files), the
run_all_tests.py wrapper, the load?transform?assert pattern (fusion_compare.load_videos), the
inline-tmp_path fixture idiom, the skip-if-corpus-absent per-video regression litmus
(test_served_gate_a_regression.py), and the committed eval_baselines/ no-regression baselines.
NOT BUILT / GAP the task targets: there are currently NO committed per-video regression fixtures
and NO parametrized test that auto-discovers committed per-video artifacts ? the existing per-
video regression test depends on the uncommitted gitignored output/ corpus, so it never runs in
CI. A new tests/fixtures/golden/ dir of small committed *_golden_features.json + a parametrized
test would make per-video regression CI-runnable for the first time. (No confest.py exists yet
either, so shared fixture plumbing would be a new addition.)",
"constraints": [
"Commercial-clean licensing + privacy: committed fixtures must EXCLUDE minors from any
training/eval pool and must not embed paid-LLM (Gemini) outputs as a training data source
(CLAUDE.md guardrails). Real-player footage frames cannot be committed; synthetic or trajectory-
only/feature-only JSON is the safe form.",
"Fixtures must be TINY and offline: the suite is fully mocked, no GPU/WSL/network, and CI
has a 70% coverage floor. Committed *_golden_features.json should be small (a handful of
candidates), like the synthetic ones in test_fusion_compare.py ? not the full real corpus.",
"CSV fixtures MUST conform to one of the two gt_loader conventions (header start,end |
start_time,end_time, or positional) or they will be rejected/skipped by load_gt_intervals
(strict mode raises with file:line).",
"File-placement rule (CODE_MAP §0): tests go in tests/test_*.py; eval baselines go in
tracked eval_baselines/; pipeline run outputs go in gitignored output/. There is no existing
tests/fixtures or tests/data dir ? adding one is a new convention but consistent with 'a test ->
tests/'. Do NOT put fixtures in output/ (gitignored, won't survive checkout/CI).",
"Branch from origin/master, stage explicit paths (no git add -A), re-verify HEAD before
commit ? shared checkout / concurrent sessions (CLAUDE.md + memory gotcha-shared-checkout-
branch-collisions).",
"MANIFEST.in / hatchling packaging: pyproject lists packages=[backend, training]; a
tests/fixtures dir is fine for pytest (run from source tree) but would not be packaged into the
wheel ? acceptable since tests aren't shipped."
],
"relevantPaths": [
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\run_all_tests.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pytest.ini",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\pyproject.toml",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_eval_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_gt_loader.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_fusion_golden_telemetry.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_compare.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_served_gate_a_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\tests\\test_per_user_regression.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_wasb_runner.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\backend\\eval\\fusion_golden.py",
"C:\\Users\\avidu\\Projects\\badminton-highlight-

```

```

indexer\\backend\\eval\\fusion_compare.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\gt_loader.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\regression.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\per_user_regression.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval\\baselines\\heuristic_lovo_n6.json",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval\\baselines\\experiment_leaderboard.json",
    "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md"
],
"risksGaps": [
    "RECOMMENDED PLACEMENT for new auto-discovered per-video fixtures: create
tests/fixtures/golden/ holding small committed *_golden_features.json (one per synthetic video,
matching the fusion_golden output schema), and a new tests/test_golden_regression.py that does
`FIXTURE_DIR = Path(__file__).parent/'fixtures'/'golden'` then
`@pytest.mark.parametrize('fixture', sorted(FIXTURE_DIR.glob('*_golden_features.json'))),
ids=lambda p: p.stem)`. This mirrors fusion_compare.load_videos (glob+json.load) but reads
COMMITTED files instead of tmp_path, and mirrors test_served_gate_a_regression.py's per-video
loop ? except it runs in CI because the fixtures are committed. Resolve paths via
Path(__file__).parent, NEVER cwd (agent cwd resets between bash calls; tests may run from any
dir).",
    "There is no conftest.py today. If the parametrized test needs a shared loader/fixture,
add tests/conftest.py (new file) with a small `load_golden_fixture` helper or a `golden_videos`
fixture ? but a module-local glob is simpler and matches the existing zero-conftest convention;
prefer that unless multiple test files need it.",
    "Do NOT reuse output/ for committed fixtures ? it is gitignored and won't survive a clean
checkout or exist in CI (that is exactly why test_served_gate_a_regression.py skips in CI). The
fixtures must be inside tests/ to be tracked and discovered.",
    "Privacy/licensing risk if real footage-derived data is committed: the real 6-video corpus
is owner footage; commit only SYNTHETIC or fully-anonymized feature JSON (no frames, no minors),
consistent with the inline-synthetic data already used in test_fusion_compare.py. If anchoring
on real numbers, store only the derived F1/baseline in eval_baselines/ (already the convention),
not raw player data.",
    "A committed-fixture regression test should assert against a committed expected value
(e.g. an eval_baselines/ entry or an inline EXPECTED constant) using the existing
regression.regress / experiment.compare / metrics.score transforms ? reuse those, do not write
new scoring math (CODE_MAP warns calibrate_wasb.load_golden_gts + experiment harness are widely-
shared APIs).",
    "Marker hygiene: no custom pytest markers are registered (no [tool.pytest] markers
config). If the new test introduces a marker (e.g. @pytest.mark.golden), register it in
pytest.ini's `markers =` or it will emit PytestUnknownMarkWarning; simpler to rely on
parametrize + skipif like the rest of the suite."
]
},
{
    "area": "Repo constraints for committing structured regression fixtures (file-placement,
gitignore, git-lfs, schema/version conventions, licensing/privacy guardrails, size budget)",
    "summary": "The repo deliberately commits NO raw/derived video data today: there are zero
tracked .csv/.jsonl/fixture files, tests/ has no data dirs (tests build synthetic data in-code),
and the ONLY committed structured data is tiny aggregate-metric JSON in eval_baselines/ (e.g.
heuristic_lovo_n6.json ? mean_f1 + per-video scores, no per-frame data). The file-placement
table (CODE_MAP $0) routes pipeline outputs (trajectories, features, DBs, frames) to gitignored
output/, and a dedicated doc ? docs/GOLDEN_DATA_SHARING.md (#175) ? explicitly states the golden
corpus is PROPRIETARY and its derived features must live in a PRIVATE OneDrive folder OUTSIDE
the repo, never committed, with a guardrail forbidding symlinking it into a tracked path. git-
lfs is INSTALLED (3.5.1, global filters) but NOT in use (no .gitattributes anywhere). On the
licensing/privacy question: committing detector-derived numeric streams (shuttle X/Y, segment
intervals, person/audio features) is defensible because the golden videos are owned/licensed by
the founder (self-recorded + ShuttleSet, never end-user uploads) and the streams are abstract
numbers, not frames ? BUT the repo's OWN convention currently says keep them out of git (private
OneDrive), and minors must be excluded, so any decision to commit even derived data is a
product/privacy call that is owner-gated, not a free engineering choice.",

```

```

"keyFindings": [
  "(1) WHERE FIXTURES BELONG: Per docs/CODE_MAP.md S0 file-placement table ? tests go in
tests/test_*.py (pytest auto-discover, 70% coverage floor, must be pure/offline/mocked, no
GPU/WSL/network). Small TRACKED metric baselines go in eval_baselines/ ('Tracked ? survives a
clean checkout; the no-regression gate reads it'). Model-artifact MANIFESTS go in
artifacts/<name>/<ver>/ (manifest committed, weights gitignored ? ADR-008). Pipeline run outputs
(reels, TRAJECTORIES, DBs, frames) go in output/ = GITIGNORED.",
  "(2) WHAT IS GITIGNORED: .gitignore ignores output/, output_dir/, scratch/, .env/, all
*.db/*.db-journal/*.db-wal, *.pt (weights), .env*, coverage, __pycache__, and specific
backend/static highlight mp4s. NOTE: artifacts/ itself is NOT in .gitignore (it currently shows
as untracked in git status) ? only model WEIGHTS inside it are meant to be gitignored per
ADR-008, the manifest is committed. scratch/ is fully gitignored+excluded from ruff/pytest.",
  "(3) GIT-LFS: git-lfs v3.5.1 is installed and its filters are registered globally
(filter.lfs.clean/smudge/process, required=true), BUT there is NO .gitattributes file anywhere
in the repo ? so LFS tracks nothing and is effectively NOT in use. LFS is AVAILABLE if needed
but the repo has chosen the 'keep big data out of git' path (output/ gitignored + private
OneDrive sync) instead of LFS.",
  "(4) NO EXISTING DATA FIXTURE PRECEDENT: `git ls-files` finds ZERO committed .csv/.jsonl
files and no tests/ data/fixture/golden directories. Tests (e.g.
test_fusion_golden_telemetry.py) build manifests + synthetic data inline in code with
monkeypatch/tmp_path. The ONLY committed structured-data precedent is eval_baselines/*.json ?
and those are tiny AGGREGATE metrics (mean_f1, std, per_video score dict), NOT raw per-frame or
per-window derived streams.",
  "(5) PROPRIETARY-DATA GUARDRAIL IS EXPLICIT: docs/GOLDEN_DATA_SHARING.md (status
2026-06-15, #175) says the GPU-extracted golden artifacts (trajectory CSVs,
*_golden_features.json, manifests) sync via a PRIVATE OneDrive folder at
%USERPROFILE%\OneDrive\rally-annotator\golden-data\ ? OUTSIDE the repo ? precisely so
large/proprietary data is 'never committed to git.' Guardrail: 'features derive from the
Proprietary corpus ? keep the OneDrive folder private (no public share links)' and 'do not
symlink the shared folder into a tracked path.'",
  "(6) SCHEMA/VERSION CONVENTIONS TO MIRROR: DB uses a PRAGMA user_version migration ladder
in backend/infrastructure/database.py (currently at v5; ordered `if version < N:` blocks,
additive ALTERs that preserve prior rows, tests assert the expected version so drift fails CI).
Detector data is DETECTOR-AGNOSTIC: common fields are columns, detector-specific metrics go in a
`stats` JSON blob keyed by a `detector` string ? a new detector reuses the table. eval_baselines
JSON carries provenance fields (name, description, `computed` repro command, metric values).
Golden features JSON has a stable shape: {name, fps, frame_width,
candidates:[{start,end,shuttle{5 feats},person{5 feats},label}], gts:[[a,b]]}. Any fixture
should likewise carry a version/schema tag + provenance and follow the column-vs-stats-JSON
split.",
  "(7) VIDEO PROVENANCE IS CLEAN-BY-DESIGN: docs/GOLDEN_SET_VENDURING.md ? 'Golden sets are
built only from footage we license or own ? starting with the founder's personal collection,
plus ShuttleSet, plus purpose-recorded clips. No end-user uploads ever flow to a vendor.' So the
source is owned/self-recorded + ShuttleSet (broadcast contrast only), NOT third-party broadcast
scraping.",
  "(8) MINORS / PRIVACY GUARDRAIL: CLAUDE.md + COMMERCIALIZATION.md C14 +
OWNED_MODEL_IMPLEMENTATION_PLAN mandate 'exclude minors' and 'per-user training data needs a
separate explicit opt-in.' FIELD_VISIT_PLAYBOOK: 'sessions with minors are not recorded for our
library ? adult sessions only.' KHELSUTRA_PRODUCT_OVERVIEW: 'Footage involving minors is never
used to improve the tool and is never published.' Any committed fixture must be guaranteed
minor-free at source.",
  "(9) LICENSING GUARDRAIL (for completeness): CLAUDE.md ? paid LLMs
(Gemini/OpenAI/Anthropic) are inference/serving ONLY, NEVER a training data source; all
code+data+weights must be MIT/Apache-2.0/BSD. Derived numeric streams from owned video do not
implicate paid-LLM-output rules (they come from MIT-licensed WASB/TrackNet detectors, not from
Gemini).",
],
"dataStructures": [
  {
    "name": "Trajectory CSV (per-video shuttle stream)",
    "where": "Produced to gitignored output/ (predict writes <stem>_predict.csv); parsed by
backend/pipeline/detectors/tracknet_runner.py::parse_trajectory_csv",
    "shapeOrFormat": "CSV header `Frame,Visibility,X,Y` ? exactly one row per video frame
(int frame index, 0/1 visibility, pixel X, pixel Y). Pure numeric, no imagery."
  },
]

```

```

{
  "name": "*_golden_features.json (per-video derived feature record)",
  "where": "backend/eval/fusion_golden.py::extract_video return value; written one JSON
per video to out_dir (output/ or the private OneDrive features/ folder)",
  "shapeOrFormat": "{name, fps, frame_width, candidates:[{start, end, shuttle:{~5
fps/resolution-invariant feats}, person:{~5 person-cue feats}, label:int}},
gts:[{start,end},...]]. The replayable CPU re-scoring substrate; small (a few KB?tens of KB).",
},
{
  "name": "eval_baselines/*.json (the ONLY committed structured-data precedent)",
  "where": "eval_baselines/heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json,
experiment_leaderboard.json ? TRACKED in git",
  "shapeOrFormat": "Tiny aggregate-metric JSON: {name, description, computed:<repro
command>, mean_f1, std, per_video:{video_name: f1, ...}}. Carries provenance; NO per-frame/per-
window data."
},
{
  "name": "candidate_segments table (detector-agnostic DB schema)",
  "where": "backend/infrastructure/database.py, migration v1 (PRAGMA user_version=1)",
  "shapeOrFormat": "Columns: id, video_id, detector(str), chunk_index, start_time,
end_time, duration, confidence, stats(JSON for detector-specific metrics),
detection_params(JSON), confirmed_segment_id, human_label, notes, user_id(v4, nullable). Common
fields=columns, detector-specific=stats JSON ? the reuse pattern a fixture should mirror."
},
{
  "name": "PRAGMA user_version migration ladder",
  "where": "backend/infrastructure/database.py::_init_db",
  "shapeOrFormat": "Ordered `if version < N:` blocks; currently at v5 (v1
candidate_segments, v2 jobs, v3 self-scheduling, v4 user_id columns, v5 user_models). Additive
ALTERs preserve rows; tests assert expected version (drift fails CI).",
}
],
"videoDependency": "High but CLEAN. The derived streams (shuttle X/Y trajectories,
candidate-window features, person/audio cues) are extracted FROM the golden videos, so fixtures
encode information about proprietary footage. However: (a) the videos are owned/licensed by the
founder (self-recorded + ShuttleSet), never third-party broadcast scraping and never end-user
uploads (GOLDEN_SET_VENDORING.md); (b) the fixtures are abstract NUMERIC streams (frame index,
pixel coords, scalar features) ? NOT frames/imagery, so they carry no biometric/identifiable
content; (c) the detectors that produce them (WASB/TrackNet) are MIT-licensed and the streams
contain no paid-LLM output, so the licensing guardrail is not implicated. The binding constraint
is the repo's OWN convention (GOLDEN_DATA_SHARING.md) that these proprietary-derived artifacts
live in private OneDrive and are NOT committed ? committing them is a product/privacy reversal
that is owner-gated.",
"builtVsPlanned": "BUILT: the gitignore rules; eval_baselines/ as the tracked-baseline
pattern; the PRAGMA user_version ladder (v5) with detector-agnostic stats-JSON tables;
backend/eval/fusion_golden.py producing the *_golden_features.json streams; the trajectory CSV
parse path; the test suite's synthetic-data-in-code pattern (no committed data fixtures).
CONVENTION/PLAN (not code-enforced): GOLDEN_DATA_SHARING.md private-OneDrive sync (#175,
status='convention + plan 2026-06-15' ? the optional scripts/sync_golden.py and RALLY_GOLDEN_DIR
env var are NOT required/built yet); the hard train/eval-leakage guardrails
(permissions/approval at ingest+export) are explicitly 'TODO ? design with the user' in
GOLDEN_SET_PHASE1_VIDEOS §5 / BACKLOG; the consent ledger (contribution_consent, minors-
exclusion at the writer) is PERSONALIZATION_PLAN v6 'pending.' git-lfs is available but unused
(no .gitattributes).",
"constraints": [
  "Committed fixtures = tests/test_*.py (pure/offline/mocked, 70% cov floor) for logic, and
eval_baselines/*.json for tracked metric baselines. Bulk derived data does NOT go in git ?
output/ is gitignored and the golden artifacts are designed to live in private OneDrive, NOT the
repo.",
  "Do NOT commit raw frames/video, *.db, *.pt weights, or large per-frame streams ? all
gitignored by convention. Model artifacts: commit the manifest in artifacts/<name>/<ver>/,
gitignore the weights (ADR-008).",
  "Mirror the detector-agnostic schema: common fields as columns/keys + a `stats`/params
JSON blob keyed by a `detector` string; carry a version/schema tag + provenance (like
eval_baselines' `computed` repro command and like PRAGMA user_version).",

```

"Licensing: every dependency (code+data+weights) MIT/Apache-2.0/BSD only; paid LLMs are serving-only and NEVER a training/data source ? derived streams must come from the MIT WASB/TrackNet detectors, not Gemini output.",

"Privacy: exclude minors at the source; per-user/contributed data needs explicit separate opt-in + consent ledger; footage involving minors is never used to improve the tool. Committing derived data from such footage would violate this even though it's 'just numbers.'",

"Branching safety (CLAUDE.md): branch from origin/master explicitly, re-verify HEAD before commit, stage explicit paths (never git add -A) ? relevant when actually landing any fixture PR. NOTE: `git pull --ff-only` failed at session start ('Cannot fast-forward to multiple branches') ? the local checkout may lag origin; treat as possibly stale."

```

],
"relevantPaths": [
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\CODE_MAP.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_DATA_SHARING.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_VENDORING.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\GOLDEN_SET_PHASE1_VIDEOS.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.gitignore",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\infrastructure\\database.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\eval\\fusion_golden.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\pipeline\\detectors\\tracknet_runner.py",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\eval\\baselines\\heuristic_lovo_n6.json",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\COMMERCIALIZATION.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\docs\\OWNED_MODEL_IMPLEMENTATION_PLAN.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\CLAUDE.md",
  "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\tests\\test_fusion_golden_telemetry.py"
],
"risksGaps": [
  "RECOMMENDED DESIGN TO STAY CLEAN: commit only (a) tiny aggregate/metric fixtures and small DERIVED NUMERIC streams (trajectory CSV rows, feature JSON) ? never frames/imagery; (b) from videos guaranteed owned/licensed AND minor-free at source; (c) with an explicit provenance/license/consent tag + a version/schema tag in each fixture. This keeps licensing clean (MIT detectors, no paid-LLM output) and privacy clean (no biometrics, no minors).",
  "POLICY CONFLICT TO RESOLVE WITH OWNER: GOLDEN_DATA_SHARING.md currently says the proprietary-derived golden features stay in PRIVATE OneDrive and are NOT committed. A 'commit a small regression fixture to git' design REVERSES that convention. Because the corpus is labeled Proprietary, this is a product/privacy decision (owner-gated), not a unilateral engineering choice ? get explicit sign-off and ideally commit a redacted/synthetic or a tiny owner-cleared subset rather than the full corpus.",
  "SIZE BUDGET: corpus is n=6 owned golden videos today (eval_baselines/heuristic_lovo_n6.json + LOVO docs). Per-video trajectory CSV = one row per frame of `Frame,Visibility,X,Y` (~20-25 bytes/row); a ~2-4 min clip at 30-60fps ? 3.6k-14.4k rows ? ~80KB-350KB per video ? full n=6 trajectory set ? <2MB. The *_golden_features.json (one record per candidate window, ~10 feats) is far smaller ? a few KB to tens of KB per video. So a numeric-only regression fixture for all 6 videos is comfortably git-sized (low single-digit MB); git-lfs/.gitattributes is NOT needed at this scale.",
  "GAP: no code-enforced guardrail yet prevents derived data from a minors/unconsented or unlicensed video being committed ? the leakage/consent guards are TODO (PHASE1_VIDEOS $5, PERSONALIZATION_PLAN v6 pending). Until those exist, the minor-free/owned guarantee rests on manual discipline; a committed fixture should carry an explicit per-video provenance+consent assertion.",
  "GAP: artifacts/ is untracked and NOT listed in .gitignore ? if a fixture-generating PR also touches artifacts/, be careful not to accidentally stage weights; stage explicit paths only.",
  "STALENESS FLAG: `git pull --ff-only` failed ('Cannot fast-forward to multiple branches') ? local master may lag origin/master; re-fetch and branch from origin/master before landing any fixture PR (per CLAUDE.md shared-checkout safety)."
]
}

```

]

2. assistant (2026-06-16T02:28:05.024Z)

I'll analyze the grounding to design the minimal video-free regression approach. Let me verify a few load-bearing details against the actual code before finalizing.

3. user (2026-06-16T02:28:06.447Z)

```
1      """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3      The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4      and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5      no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6      designed seam): the person features are extra columns the learned LogisticRegression
weights,
7      so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9      Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19
20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cands = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cands]
```

```

49         gts = [(float(a), float(b)) for a, b in d["gts"]]
50         videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                     features=features, gts=gts))
52     return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                 threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
58     feature_names=list(SHUTTLE_FEATURE_NAMES),
59     threshold=threshold)
60         fusion = experiment.Variant(name="shuttle_person_fusion",
61     feature_names=list(SHUTTLE_FEATURE_NAMES) +
62     list(PERSON_FEATURE_NAMES),
63     threshold=threshold)
64         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66     def format_result(res: Dict[str, Any], iou: float) -> str:
67         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68         ds = delta_stats(res)
69         st = ds["sign_test"]
70         lines = [
71             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
72             ===",
73             f"    shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
74             IoU={a['mean_iou']:.3f}",
75             f"    +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
76             IoU={b['mean_iou']:.3f}",
77             f"    ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
78             ?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
79             f"    paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
80             {ds['ci_high']:+.3f}]  "
81             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
82             f"| CI clears 0: {ds['ci_excludes_zero']}",
83             f"    per-video ?F1:",
84             ]
85         for pv in res["per_video_delta"]:
86             lines.append(f"        {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
87         return "\n".join(lines)
88
89
90     def main() -> None:
91         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
92 shuttle+person (golden set).")
93         ap.add_argument("--features-dir", default="output")
94         ap.add_argument("--iou", type=float, default=0.5)
95         ap.add_argument("--record", action="store_true", help="append the result to the
96 experiment leaderboard")
97         args = ap.parse_args()
98
99         videos = load_videos(args.features_dir)
100         if len(videos) < 2:
101             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
102 {len(videos)})")
103         res = compare(videos, iou=args.iou)
104         print(format_result(res, args.iou))
105         if args.record:
106             row = experiment.record_experiment(res)
107             print(f"recorded -> eval_baselines/experiment_leaderboard.json
108 ({row['timestamp']}")
109
110
111 if __name__ == "__main__":
112     main()

```

4. user (2026-06-16T02:28:06.927Z)

```

1      """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOVO).
2
3      The LOVO driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4      non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5      report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6      """
7      import json
8      import os
9      from types import SimpleNamespace
10
11     import pytest
12
13     from backend.eval import fusion_compare
14     from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cand = []
21         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0), ((50.0,
51.0), 0)]:
22             shuttle = {fn: 1.0 for fn in SHUTTLE_FEATURE_NAMES}          # constant
-> non-separating
23             person = {fn: 0.0 for fn in PERSON_FEATURE_NAMES}
24             person["players_in_court"] = 4.0 if pos else 0.0              # the
separating signal
25             person["in_court_ratio"] = 1.0 if pos else 0.0
26             cand.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
"label": pos})
27             data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
28                     "candidates": cand, "gts": [[0.0, 10.0], [20.0, 30.0]]}
29             with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
30                 json.dump(data, f)
31
32
33     def test_fusion_compare_reports_person_driven_lift(tmp_path):
34         d = str(tmp_path)
35         for k in range(6):
36             _write_video(d, f"vid{k}")
37         videos = fusion_compare.load_videos(d)
38         assert len(videos) == 6
39         res = fusion_compare.compare(videos, iou=0.5)
40         assert res["num_videos"] == 6
41         assert res["variant_b"]["honest_lovo"] is True                    # LOVO-honest learned
variant
42         # Fusion (with the separating person feature) must beat shuttle-only (no signal).
43         assert res["delta"]["f1"] > 0.0
44         assert res["variant_b"]["aggregate"]["f1"] > res["variant_a"]["aggregate"]["f1"]
45         # format_result must not crash and must mention both variants.
46         out = fusion_compare.format_result(res, 0.5)
47         assert "shuttle-only" in out and "+person" in out
48
49
50     def test_fusion_compare_delta_stats_and_ci(tmp_path):
51         """delta_stats yields a paired bootstrap CI + sign test; format_result reports
them."""

```



```

52     d = str(tmp_path)
53     for k in range(6):
54         _write_video(d, f"vid{k}")
55     res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
56     ds = fusion_compare.delta_stats(res)
57     assert set(ds) >= {"mean_delta", "ci_low", "ci_high", "sign_test",
"ci_excludes_zero"}
58     assert ds["ci_low"] - 1e-9 <= ds["mean_delta"] <= ds["ci_high"] + 1e-9    # CI
brackets mean ( $\pm$ fp)
59     assert ds["sign_test"]["wins"] == 6                # fusion wins on every video (person
feature separates)
60     assert ds["ci_excludes_zero"] is True
61     out = fusion_compare.format_result(res, 0.5)
62     assert "CI" in out and "sign test" in out
63
64
65     def test_fusion_compare_needs_two_videos(tmp_path):
66         _write_video(str(tmp_path), "only")
67         videos = fusion_compare.load_videos(str(tmp_path))
68         assert len(videos) == 1 # compare() / experiment.compare needs >=2 for honest LOVO
69
70
71     # ----- extractor assembly
72
73     def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74         from backend.eval import fusion_golden
75         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
76
77         intervals = [(0.0, 10.0), (40.0, 41.0)]
78         x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79         points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))
80                     for i in range(10)]
81
82         monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                             lambda traj, fps, fw: (intervals, x_shuttle))
84         monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
85         monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
86
87         class _StubDetector:
88             def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
89                 return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
90                             "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
91                         for (sf, ef) in windows]
92
93         spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
94                "labels": "foo.rallies.csv"}
95         data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)
96
97         assert data["name"] == "t"
98         assert data["gts"] == [(0.0, 10.0)]
99         assert len(data["candidates"]) == 2
100        c0 = data["candidates"][0]
101        assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
102        assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
103        assert c0["label"] == 1                # (0,10) matches the gt -> positive
104        assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
105        assert all(isinstance(v, float) for v in c0["person"].values())
106
107
108     def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
109         """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""

```

```

110     from backend.eval import fusion_golden
111     out_dir = str(tmp_path / "out")
112     os.makedirs(out_dir)
113     manifest = [
114         {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
115         {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
116     ]
117     manifest_path = str(tmp_path / "m.json")
118     with open(manifest_path, "w", encoding="utf-8") as f:
119         json.dump(manifest, f)
120     # a prior run already produced alpha's JSON
121     with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
encoding="utf-8") as f:
122         json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)
123
124     extracted = []
125
126     def _stub(spec, **kw):
127         extracted.append(spec["name"])
128         return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
129
130     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
131
132     fusion_golden.run(manifest_path, out_dir, fresh=False)
133     assert extracted == ["beta"] # alpha skipped (already extracted)
134
135     extracted.clear()
136     fusion_golden.run(manifest_path, out_dir, fresh=True)
137     assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140     def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141         """--smallest-first orders by trajectory size; default keeps manifest order."""
142         from backend.eval import fusion_golden
143         out_dir = str(tmp_path / "out")
144         os.makedirs(out_dir)
145         (tmp_path / "big.csv").write_text("x" * 2000)
146         (tmp_path / "small.csv").write_text("x" * 20)
147         manifest = [
148             {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
149             {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
150         ]
151         mp = str(tmp_path / "m.json")
152         with open(mp, "w", encoding="utf-8") as f:
153             json.dump(manifest, f)
154
155         order = []
156
157         def _stub(spec, **kw):
158             order.append(spec["name"])
159             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
160
161         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
162
163         fusion_golden.run(mp, out_dir, smallest_first=True)
164         assert order == ["small", "big"] # smaller trajectory first
165
166         order.clear()

```

```

167         fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
168         assert order == ["big", "small"]          # manifest order preserved
169
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
173         assert _derive_video_path({"labels": "X/Adarsh and Avi.rallies.csv"}) == "X/Adarsh
and Avi.MP4"
174         assert _derive_video_path({"video_path": "a.mp4", "labels": "b.rallies.csv"}) ==
"a.mp4"
175         with pytest.raises(ValueError):
176             _derive_video_path({"name": "n", "labels": "weird.csv"})
177

```

5. user (2026-06-16T02:28:07.462Z)

```

1      {
2          "name": "heuristic_velocity_windowing_LOVO",
3          "description": "The honest cross-video floor the Gen-0 learned model must beat:
calibrated velocity-windowing heuristic, leave-one-video-out over the n=6 owned golden corpus.",
4          "computed": "2026-06-10, reproduced via: python -m backend.eval.calibrate_local
--manifest output/human_lovo_manifest.json --metric f1 (matches docs/archives/quality-
iterations/2026-06-multivideo-lovo)",
5          "mean_f1": 0.480,
6          "std": 0.242,
7          "per_video": {
8              "adarsh_avi_singles": 0.751,
9              "kushagra_singles": 0.157,
10             "largetest_doubles": 0.737,
11             "gbaaddy": 0.309,
12             "testlarge_short": 0.267,
13             "mahadevpura_singles": 0.657
14         }
15     }
16

```

6. assistant (2026-06-16T02:28:12.587Z)

The grounding is thorough and consistent. I have what I need: the seam (the ``*_golden_features.json`` substrate consumed by ``fusion_compare.load_videos`` / ``experiment.compare``), the proven committed home (``eval_baselines/``), and the no-data-fixtures-under-tests convention. Let me verify the two remaining load-bearing facts ? that ``experiment.compare`` is deterministic on a fixed input and the ``regression.py`` baseline/exit-code contract ? then finalize.

7. user (2026-06-16T02:28:14.003Z)

```

1      """Regression runner ? verify the production tool against golden-set ground truth.
2
3      Implements the explicit ask: *"given a video, get a golden set created and verify
4      its precision and recall against the production tool."* This is deliberately a thin
5      wrapper over machinery that already exists:
6
7      ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8      predictions   ? the production pipeline's stored DB rows    [backend.eval.harness]
9      comparison    ? interval matching + P/R/F1                  [backend.eval.metrics]
10     baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12     A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13     baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14     later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15     independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16     """
17     import os

```

```

18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,

```

8. user (2026-06-16T02:28:15.230Z)

```
25: - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26: aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
102: (honest) ? the recipe, not a pinned artifact.
283: evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`.
297: iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
303: so no leakage; per-video scoring is already honest.
307: - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO ?
309: - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
322: if recipe_lovo and not honest:
328: if recipe_lovo and honest:
331: raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a pinned
model")
336: elif recipe_lovo: # honest=False ? one model over all videos
352: "honest_lovo": recipe_lovo and honest,
360: """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare mean)."""
384: def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
386: """C1 + C4 honest reporting layered over a compare() pair (additive, EXPERIMENT_HARNESS
$6).
397: "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
403: def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404: iou: float = 0.5, honest: bool = True, honest_stats: bool = True) -> Dict[str,
Any]:
409: learned variants are scored LOVO-honestly. The result is a self-contained record fit for
412: `honest_stats` (default True) **adds** a `honest` block ? per-metric bootstrap
CIs, a
415: shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
$13/C1+C4): a
420: eval_a = evaluate_variant(videos, variant_a, iou, honest)
421: eval_b = evaluate_variant(videos, variant_b, iou, honest)
446: if honest_stats:
447: result["honest"] = honest_summary(videos, eval_a, eval_b)
454: def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
524: # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries the
526: honest = result.get("honest") or {}
527: if honest.get("paired_delta_f1"):
528: row["honest_delta_f1"] = honest["paired_delta_f1"]
```

9. assistant (2026-06-16T02:28:19.252Z)

The bootstrap CI in `paired\_delta\_ci` introduces non-determinism unless seeded ? that's a key constraint for the regression test design. The deterministic core to assert on is the variant scoring (P/R/F1/mIoU per video, the LOVO `evaluate\_variant` output), not the bootstrap CI. Let me confirm the bootstrap seeding behavior, since it decides whether the regression test asserts on the score path only or can include the CI.

10. user (2026-06-16T02:28:21.297Z)

```
18: deterministic (bootstrap takes an explicit `seed`) ? nothing here changes existing
24: import random
32: def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33: n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34: """Percentile bootstrap CI for the mean of `values`. Deterministic given `seed`.
45: rng = random.Random(seed)
47: for _ in range(max(1, n_boot)):
58: n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
64: lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
90: def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91: confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
101: summary = mean_with_ci(deltas, confidence, n_boot, seed)
```

11. assistant (2026-06-16T02:28:27.658Z)

The bootstrap is deterministic with a default `seed=0`. So the entire `experiment.compare` output ? per-video P/R/F1, LOVO aggregate, paired ?F1 CI, and sign test ? is fully deterministic given a fixed set of committed `*_golden_features.json` fixtures. That confirms the whole chain can be asserted exactly (the only float-equality caveat is to assert with a small tolerance, which the existing tests already do).

I now have everything needed. The design writes itself from the grounding: the seam is already operational in eval, the engine (`experiment.compare`) is pure and deterministic, the committed home (`eval_baselines/`) is proven, and the only missing pieces are (a) committed fixtures, (b) a frozen expected-scores baseline, (c) a CI test, and (d) an auto-grow step that the already-built `fusion_golden.py` produces.

12. user (2026-06-16T02:30:11.820Z)

Structured output provided successfully

13. assistant (2026-06-16T02:30:20.565Z)

I've completed the design analysis and returned it via StructuredOutput.

Key verified facts that shaped the design:

- The video-free regression engine already exists and is deterministic: `backend/eval/fusion_compare.py::load_videos` globs `*_golden_features.json` and `backend/eval/experiment.py::compare` scores them with seeded bootstrap (`eval_stats` default `seed=0`) and pure interval math (`metrics.py`). Only the committed inputs are missing.
- The freeze seam to pick is the per-video `*_golden_features.json` substrate (scale/translation-invariant scalars + GT intervals, no frames) ? downstream of the GPU passes, upstream of all pure scoring.
- The proven tracked home is `eval_baselines/` (survives a clean checkout); `tests/` has zero committed data fixtures today, so reusing `eval_baselines/golden_fixtures/` keeps the existing convention.
- Auto-grow is free: the same already-built `backend/eval/fusion_golden.py` the owner runs per golden video emits the exact committed shape, so adding a video is copy-file + re-snapshot.
- Net-new code is minimal: one snapshot JSON, a ~40-line parametrized `tests/test_golden_regression.py`, and a ~15-line `--snapshot` flag on `fusion_compare.py`.

Load-bearing caveat flagged in the design: committing per-video derived features reverses the current `docs/GOLDEN_DATA_SHARING.md` private-OneDrive convention, so it is owner-gated ? the shippable-now form is a synthetic or owner-cleared minor-free subset (matching the synthetic fixtures already in `tests/test_fusion_compare.py::write_video`).